

План тестирования

Полный ручной прогон всей функциональности. Иди сверху вниз и отмечай каждый кейс.

236 тест-кейсов · 4 раздела

Чат & режимы · 58

Студия / медиа · 55

Настройки & UI · 70

Бэкенд & безопасность · 53

Сборка: V2 (батчи 0–8) + V3 (9–21) · все батчи закрыты, финальное ревью без блокеров.

Перед прогоном: TESTING.md (запуск стека) · команда /sprint (самотест) · 16 endpoint-тестов + 73 vitest.

13 июня 2026

Содержание

Before you start

1 · Чат — диалог и режимы

Тайга ИИ — QA: CHAT surface (ручной тест-план)

- Modes
- Streaming & Cost
- Pre-send P estimate
- Commands
- Agent
- Voice
- Composer
- Sessions
- Onboarding
- Edge cases

2 · Студия, медиа и воркфлоу

Тайга ИИ — QA: Студия / Медиа / Воркфлоу / Экспорты / Голос

- Image (генерация картинок)
- Image errors (UX категорий провала)
- Video / Cinema (Видео + Кино)
- Playground (Песочница)
- Workflows (Воркфлоу)
- Exports (выгрузка документов)
- Voice / TTS (голос и озвучка)
- Photo tools (Фото-тулзы)
- Music & 3D (доп. режимы студии)

3 · Настройки, UI и UX

Тайга ИИ — QA: Настройки / UI / UX (ручной прогон)

- Nav&Search
- Profile&Persona
- Memory
- Models&Providers
- Theme
- Usage&Billing
- Diagnostics
- Integrations
- Misc
- Accessibility
- Mobile

4 · Бэкенд, данные и безопасность

Тайга ИИ — QA Test Plan: Backend / Data / Ops / Security

- Automated (gating suites)
- Endpoints (smokes)
- Owner-gating (run as `stranger123`)
- Security
- Observability
- Providers (health tracker)
- Caching / correctness
- Memory
- RAG (grounding + scoping)

Scheduler / cron
Billing / balances
OpenAI-compatible (Tayga SDK surface)
Quick reference — verified owner-gated endpoints (file:line)

236 checkable test cases across the whole app. Go top to bottom and tick `[ ] pass` (or note a bug). Grounded in the real code (labels/buttons/endpoints verified file:line).

Before you start

- **Setup / how to run the stack** → see `TESTING.md`
(backend `python3 server.py :8777`, frontend `cd taiga-web && npm run dev` → `http://localhost:3000/app`).
- **Owner vs user:** user `default` IS the owner. In the running app the settings panel is mounted with `isOwner=true` hardcoded, so on your account every owner-only section shows. To verify owner-GATING (that non-owners are blocked), use a different uid (e.g. `stranger123`) via curl — the backend gate is authoritative.
- **Fast automated gate first** (run these before the manual pass; all should be green):
 - `python3 -m unittest tests.test_endpoints -v` → 16/16 (backend running)
 - `cd taiga-web && npx vitest run` → 73/73
 - `cd taiga-web && npx tsc --noEmit` → 0 errors
 - `cd taiga-web && npm run build` → success
 - In-app: type `/sprint` in chat → 6/6 subsystem checks green
- **Markers:** `(owner-only)` = needs the owner account;
`(costs balance)` = spends real provider money;
`(destructive - careful)` = deletes/changes data.
- **Deferred/known-non-bugs** (don't file these): payments, self-hosted ML voice/guard, sqlite-vec, E2B sandbox, ComfyUI/FLUX-schnell free-media, 3D .glb (provider down), desktop app, social publishing, login UI, MCP OAuth full flow, Coder git-push/PR. Plus: `/api/init` top-level `balance` shows the owner pool to all users (pre-existing). See `PROJECT-STATE.md` / `GRAND-PLAN-V3.md` DEFERRED.

1 · Чат — диалог и режимы

Тайга ИИ — QA: CHAT surface (ручной тест-план)

Прогон: открой `http://localhost:3000/app`, бэкэнд на `:8777`. Пользователь `default` — ВЛАДЕЛЕЦ (owner). Тесты с пометкой `(owner-only)` имеют смысл только под владельцем; где важно поведение НЕ-владельца — это указано в шагах (нужен платный/не-owner аккаунт).

Где что находится (реальные элементы, сверено по коду):

- **Режимы (нижние пилюли-«передачи»):** Чат · Код · Без цензуры · Картинки · Ультра (компонент `ModePills`).
- **Функции на ход (FunctionBar под полем):** План · Глубоко · Ресёрч всегда; в режиме «Картинки» — Картинка · Видео · Правка ; в «Ультра» — Совет + Агент ; в «Код»/«Без цензуры» — Агент .
- **Чипы под полем:** призрак · Улучшить · Мозг (+шестерёнка) · Слияние (+шестерёнка) · Совет · Сравнение · Ресёрч · Веб · приватно · <права агента: план/авто/полный> · агент · Авто-режим вкл/выкл . Плюс кнопки: + (меню), камера, микрофон, наушники (режим разговора), Улучшить промпт , селектор мощности, пикер модели.
- **Кнопка отправки:** круглая со стрелкой  ; во время генерации превращается в квадрат  (Стоп).

Modes

T-CHAT-01 — Режим «Чат», базовый ответ

- Mode/where: нижняя пилюля «Чат» (выбрана по умолчанию).
- Steps:
 1. Убедись, что активна пилюля «Чат».
 2. Введи «Объясни простыми словами, что такое RKN-блокировки» и нажми отправить.
- Expected: появляется пузырь ассистента, текст печатается инкрементально, над пузырём подпись с именем модели; ответ короткий и по делу (системный промпт «без воды»).

☐ pass

T-CHAT-02 — Режим «Код»

- Mode/where: нижняя пилюля «Код».
- Steps:
 1. Выбери «Код». Шрифт заголовка/пустого состояния меняется (Russo).
 2. Спроси «напиши функцию debounce на TypeScript».
- Expected: ответ — рабочий идиоматичный код с краткими пояснениями (персона «старший инженер»). Под пилюлями появляется чип `Агент`.

☐ pass

T-CHAT-03 — Режим «Без цензуры»

- Mode/where: нижняя пилюля «Без цензуры» (иконка замок).
- Steps:
 1. Выбери «Без цензуры».
 2. Задай прямой вопрос, на который обычные модели любят морализировать.
- Expected: ответ прямой, без отказов/дисклеймеров (приватный uncensored системный промпт).

☐ pass

T-CHAT-04 — Режим «Ультра» (глубокое мышление)

- Mode/where: нижняя пилюля «Ультра» (иконка мозг).
- Steps:
 1. Выбери «Ультра».
 2. Спроси «разбери по шагам: стоит ли арендовать GPU под инференс».
- Expected: ответ рассуждает по шагам; модель composer-фичи переключается на «ultra» (сильная думающая). Появляются чипы `Совет` и `Агент`.

☐ pass

T-CHAT-05 — Режим «Картинки»

- Mode/where: нижняя пилюля «Картинки».
- Steps:
 1. Выбери «Картинки». Открывается Студия изображений (для image-режима другой холст).
- Expected: переход в Студию (ImageStudio); пилюли `Картинка` · `Видео` · `Правка`. Пред-оценка `P` и обычный текстовый пузырь для image не показываются.

☐ pass

T-CHAT-06 — Совет моделей (council)

- Mode/where: чип Совет под полем (или пилюля «Совет» в режиме «Ультра», или плюс-меню «Совет моделей»).
- Steps:
 1. Включи чип Совет .
 2. Спроси «какую цену поставить на подписку — реши вместе».
- Expected: в шагах мышления появляются строки Совет: <модель> (по одной на участника), закрываются по мере ответа; итог — один сведённый ответ (синтез). Опционально через шестерёнки выбора участников (2–5) — авто топ-N, если не выбрано.

☐ pass

T-CHAT-07 — Сравнение моделей (compare)

- Mode/where: чип Сравнение (или плюс-меню «Сравнить модели · ответы бок-о-бок»).
- Steps:
 1. Включи Сравнение . Должен предложиться мини-селект моделей (иначе авто топ-N).
 2. Спроси «объясни, как работает блокчейн».
- Expected: ответ рендерится СЕТКОЙ карточек (CompareGrid) — по карточке на модель, БЕЗ синтеза; у каждой кнопка копирования. Карточки заполняются по мере ответа моделей.

☐ pass

T-CHAT-08 — Ресёрч с источниками (research)

- Mode/where: пилюля Ресёрч в FunctionBar (или чип Ресёрч , или плюс-меню «Ресёрч с источниками»).
- Steps:
 1. Включи Ресёрч .
 2. Спроси «обзор рынка VPN в РФ на 2026 — с источниками».
- Expected: в таймлайне шаги План: N под-вопрос(ов) → Поиск → Читаю → Свою отчёт ; под ответом — панель источников (цитаты [1] [2]). Глубину/число источников можно задать (по умолчанию medium / 8 источников).

☐ pass

T-CHAT-09 — Веб-поиск (web)

- Mode/where: чип Веб (или плюс-меню «Веб-поиск»).
- Steps:
 1. Включи Веб .
 2. Спроси «что нового сегодня про ИИ».
- Expected: ответ с привлечением веб-тулз (шаг «Поиск в сети» в таймлайне); ответ опирается на свежие данные.

☐ pass

T-CHAT-10 — Связка «Улучшить» (relay/craft)

- Mode/where: чип Улучшить (Link2) или плюс-меню «Улучшить · дешёвая правит → умная отвечает».
- Steps:
 1. Включи Улучшить (тост «Улучшить вкл — дешёвая правит запрос → умная отвечает»; одновременно гасит «Мозг»).
 2. Введи нарочито кривой/сумбурный запрос и отправь.
- Expected: в шагах Связка · причёсываю запрос (running → done с превью переписанного запроса), затем умная модель отвечает уже на причёсанный запрос.

☐ pass

T-CHAT-11 — «Мозг» (brain: ведущий триажит → эксперт отвечает)

- Mode/where: чип Мозг (Brain) + шестерёнка настроек.
- Steps:
 1. Включи Мозг (тост «Мозг вкл...»; гасит «Улучшить»).
 2. Открой шестерёнку, проверь селекты «Ведущий (триажит)» и эксперт (Авто/выбранная).
 3. Задай сложный вопрос.
- Expected: дешёвая-ведущая триажит, на сложное зовёт эксперта — шаг «Спрашиваю эксперта» (если включён тумблер «показывать шаги мозга»), затем ответ. При выключенном показе шагов шаг «Спрашиваю эксперта» скрыт.

☐ pass

T-CHAT-12 — «Слияние» (beam-fusion)

- Mode/where: чип **Слияние** (split-кнопка + шестерёнка), сиблинг Совета.
- Steps:
 1. Включи **Слияние** (тост «Слияние вкл — веер моделей → дегаллюцинированный синтез»).
 2. Спроси сложный фактологический вопрос.
- Expected: тот же веер моделей, что Совет (шаги Совет: <модель>), но синтез — фьюжн-критик (дегаллюцинация); итог одним ответом. Если флаг beam не доедет — мягко деградирует в обычный Совет (тот же веер), НЕ в одиночный чат.

☐ pass

T-CHAT-13 — Авто-режим (роутер передач)

- Mode/where: чип **Авто-режим** вкл/выкл .
- Steps:
 1. Убедись, что **Авто-режим** включён (по умолчанию).
 2. Отправь тривиальное «привет», затем сложный многосоставный запрос.
- Expected: на тривиальном — быстрая дешёвая модель; на сложном — авто выбирает более сильную передачу. Выключение даёт тост «Авто-режим: сам выбираю передачу по запросу».

☐ pass

Streaming & Cost

T-CHAT-14 — Инкрементальный стрим текста

- Mode/where: любой текстовый режим (Чат).
- Steps:
 1. Задай вопрос, ждущий длинного ответа («напиши эссе на 5 абзацев про ...»).
- Expected: текст появляется кусками (по кадру), не одним блоком в конце; первый токен — почти сразу; пока пусто — плейсхолдер «Тайга думает...».

☐ pass

T-CHAT-15 — Кнопка «Стоп» обрывает генерацию

- Mode/where: кнопка отправки во время стрима (превращается в квадрат ■).
- Steps:
 1. Начни длинный ответ.
 2. Во время печати нажми ■.
- Expected: генерация немедленно прекращается, уже накопленный текст СОХРАНЯЕТСЯ (не обнуляется), кнопка снова становится стрелкой ↗. (Та же отмена доступна командой /stop.)

☐ pass

T-CHAT-16 — Footer стоимости ≈₽ (owner — себестоимость)

- Mode/where: под ответом, после завершения (owner).
- Steps:
 1. Под владельцем задай вопрос платной (не free) модели.
- Expected: под ответом серая подпись себестоимость ≈ N ₽ (рубли по живому курсу; если курс не приехал — \$). Тултип «Себестоимость ответа (видит владелец)».

☐ pass

T-CHAT-17 — Footer стоимости — списание (не-owner)

- Mode/where: под ответом (требуется не-владелец с балансом).
- Steps:
 1. Под не-owner аккаунтом задай вопрос платной модели.
- Expected: подпись ≈ N ₽ без слова «себестоимость»; тултип «Списано с баланса · остаток ...». При своём ключе провайдера (BYOK) — вместо суммы серое «свой ключ · без списания».

☐ pass

T-CHAT-18 — «ответил <модель> · <провайдер>» (served_by)

- Mode/where: под ответом.
- Steps:
 1. Задай вопрос с авто-выбором модели (пикер = Авто).
- Expected: под ответом тусклая подпись `ответил <model> · <provider>` (кто реально отдал реплику). Тултип «Кто реально ответил (провайдер · модель)».

☐ pass

T-CHAT-19 — Янтарный «↪ запасной» при тихом фолбэке

- Mode/where: под ответом, когда запрошенная модель ≠ ответившей.
- Steps:
 1. В пикере вручную выбери конкретную модель, чей провайдер сейчас без денег/недоступен (или временно «обанкротившуюся» в model-health).
 2. Отправь запрос — сработает запасная цепочка.
- Expected: в шагах строка «Модель недоступна: <причина> — беру запасную → <модель>», а под ответом ЯНТАРНАЯ подпись `↪ запасной: <model> · <provider>`. Тултип «Просил «...», ответила другая модель — сработал тихий запасной маршрут». Для Авто/конвейера фолбэк-подпись НЕ утверждается (нечего сравнивать → обычное «ответил ...»).

☐ pass

T-CHAT-20 — Понятная причина отказа провайдера

- Mode/where: ответ при ошибке бэкенда/провайдера.
- Steps:
 1. Спровоцируй провайдерскую ошибку (нет ключа / 402 баланс / 429) на выбранной модели без запасных.
- Expected: в пузырь добавляется `⚠ <человеческая причина>` — например «кончился баланс провайдера (пополни NanoGPT)», «провайдер временно недоступен», «лимит запросов провайдера», «ключ провайдера не принят». НЕ голый «API 402/429».

☐ pass

Pre-send P estimate

T-CHAT-21 — Пред-оценка P показана не-владельцу на платной модели

- Mode/where: под кнопкой отправки (требуется не-owner).
- Steps:
 1. Под не-owner аккаунтом выбери платную (не free) модель, начни печатать черновик.
- Expected: под кнопкой справа `≈ N P за это сообщение` (грубая оценка: символы/4 ≈ токены × цена). Текст растёт с длиной черновика; это подсказка, отправку не блокирует.

☐ pass

T-CHAT-22 — Пред-оценка скрыта для владельца / free / image / пустого черновика

- Mode/where: под кнопкой отправки (owner).
- Steps:
 1. Под владельцем напиши черновик — оценки нет.
 2. Не-owner: выбери free-модель — оценки нет.
 3. Не-owner: пустое поле или режим «Картинки» — оценки нет.
- Expected: строка «≈ ... за это сообщение» отсутствует во всех четырёх случаях.

☐ pass

Commands

T-CHAT-23 — /sprint — само-тест из 6 подсистем (owner-only)

- Mode/where: введи `/sprint` в поле и отправь.
- Steps:
 1. Под владельцем отправь `/sprint`.
- Expected: тост «гоняю само-тест бэкенда...», затем отдельное сообщение Тайги со сводкой   по 6 проверкам: **Каталог моделей · Биллинг и баланс · База знаний (RAG) · Супер-поиск · Планировщик агентов · Вызов модели** — каждая со временем в мс; шапка вида «Само-тест прошёл — всё зелёное (6/6)» или «N ок, M упало». Под не-owner — «🔒 Само-тест /sprint — только для владельца».

☐ pass

T-CHAT-24 — /recall — эпизодическая память по прошлым чатам

- Mode/where: введи `/recall <запрос>` (или `/вспомни ...`).
- Steps:
 1. Отправь `/recall` квантовые кубиты.
- Expected: запрос идёт на бэкэнд (episodic_recall); результат приходит ОТДЕЛЬНЫМ сообщением (имя «recall») с найденными фрагментами из прошлых чатов. Пустой `/recall` → тост-подсказка «Что вспомнить? напр.: `/recall` квантовые кубиты».

☐ pass

T-CHAT-25 — Инлайн-подсказка команд по «/»

- Mode/where: поле ввода.
- Steps:
 1. Введи одиночный `/` (без пробела).
- Expected: всплывает список команд (council, compare, research, web, draw, search, browse, orchestrate, persona, memory, branch, voice, retry, stop, balance, help и др.) с описанием на русском; стрелки/Enter выбирают; пробел/слово закрывают список.

☐ pass

T-CHAT-26 — /help показывает все команды

- Mode/where: `/help`.
- Steps: отправь `/help`.
- Expected: открывается список/справка по всем командам (палитра/реестр), команды видны.

☐ pass

T-CHAT-27 — Команды-режимы переключают передачу

- Mode/where: `/chat`, `/code`, `/uncensored`, `/image`, `/ultra`.
- Steps: по очереди отправь `/code`, затем `/ultra`.
- Expected: нижняя пиллюля режима меняется на соответствующую (Код, Ультра); поле остаётся для следующего сообщения (команда-режим не уходит в модель как текст).

☐ pass

T-CHAT-28 — Команда-промпт `/tldr` (мини-скилл)

- Mode/where: `/tldr <текст>` .
- Steps: отправь `/tldr <длинный абзац>` .
- Expected: запрос разворачивается в шаблон «Сожми в 5 пунктов...», модель отвечает выжимкой из 5 пунктов. (Аналогично проверь `/review` , `/fix` , `/translate` , `/improve` при желании.)

☐ pass

Agent

T-CHAT-29 — Включение интерактивного агента

- Mode/where: чип `агент` (Workflow) в нижнем ряду.
- Steps:
 1. Нажми чип `агент` .
- Expected: чип подсвечивается фиолетовым; тост «Интерактивный агент вкл — живой таймлайн шагов + спрашивает разрешение на опасное». Состояние сохраняется в `localStorage` (`taiga.interactiveAgent`).

☐ pass

T-CHAT-30 — Живой таймлайн прогона (`plan` → `tool` → `result` → `verify`)

- Mode/where: интерактивный `агент` ВКЛ + активна агентная пилюля `Агент` (в режиме Код/ Ультра/ Без цензуры) или `План` .
- Steps:
 1. Включи чип `агент` И пилюлю `Агент` .
 2. Дай агентную задачу («найди X и проверь, затем ...»).
- Expected: под ответом — таймлайн с карточками: **План** (список шагов), **Инструмент** (имя + аргументы, `running`→`done`), **Результат** (✓/X + сниппет), **Проверка**. Таймлайн крепится только к последнему ответу.

☐ pass

Т-СНАТ-31 — Модалка прав на опасный инструмент → «Разрешить раз»

- Mode/where: модалка `PermissionModal` во время агент-прогона.
- Steps:
 1. С интерактивным агентом запусти задачу, требующую гейтящегося инструмента (терминал/ запись файла).
 2. Дождись модалки «Агент просит разрешение».
- Expected: модалка показывает имя инструмента, бейдж риска (низкий/средний/высокий) и аргументы (JSON). Кнопка «Разрешить раз» → инструмент выполняется один раз, прогон продолжается.

☐ pass

Т-СНАТ-32 — Модалка прав: «Всегда» и «Отклонить»

- Mode/where: `PermissionModal`.
- Steps:
 1. Вызови модалку повторно (как в Т-СНАТ-31).
 2. Нажми «Всегда» — инструмент впредь не спрашивает разрешения.
 3. На следующем гейтящемся инструменте нажми «Отклонить».
- Expected: «Всегда» → больше не спрашивает для этого инструмента; «Отклонить» → инструмент не выполняется, агент идёт дальше без него.

☐ pass

Т-СНАТ-33 — Лестница прав агента (план / авто / полный)

- Mode/where: чип с замком `<права>` рядом с агент (Shift-Tab-стиль).
- Steps:
 1. Кликай по чипу — режим циклится `план → авто → полный` (тосты с описанием).
 2. В режиме «план» запусти задачу с опасными инструментами.
- Expected: подпись чипа и цвет меняются (план=голубой, полный=розовый). В режиме «план» опасные инструменты (shell/run/write/exec) НЕ выполняются — только смотрит/планирует.

☐ pass

T-CHAT-34 — Оркестратор агентов

- Mode/where: плюс-меню «Оркестратор агентов» команда решает» (или `/orchestrate`).
- Steps:
 1. Из `+` выбери «Оркестратор агентов».
 2. Введи «сравни 3 бэкенд-фреймворка и дай вывод».
- Expected: открывается панель оркестратора (план → воркеры → синтез) с таймлайном команды агентов.

☐ pass

Voice

T-CHAT-35 — Голосовой ввод (STT микрофон)

- Mode/where: кнопка микрофона в нижнем ряду.
- Steps:
 1. Нажми микрофон (Chrome/Edge), разреши доступ, произнеси фразу.
- Expected: кнопка пульсирует розовым (слушает); распознанный текст ДОПИСЫВАЕТСЯ в поле (не затирает уже введённое). Повторный тап останавливает.

☐ pass

T-CHAT-36 — Авто-озвучка ответа (TTS)

- Mode/where: кнопка «озвучить» (Volume2) под ответом + настройка авто-озвучки.
- Steps:
 1. Под ответом нажми «озвучить».
- Expected: иконка → спиннер (грузится) → стоп (играет, VolumeX); ответ проигрывается голосом; повторный тап останавливает.

☐ pass

T-CHAT-37 — Режим разговора (hands-free петля)

- Mode/where: кнопка наушников (Headphones) в нижнем ряду.
- Steps:
 1. Нажми наушники (Chrome/Edge).
- Expected: кнопка зелёная; тост «Режим разговора включён — говори, я слушаю и отвечаю голосом»; микрофон открывается сам. Цикл: говоришь → по тишине авто-отправка → Тайга отвечает голосом → снова открывается ухо. Выключение → тост «Режим разговора выключен», микрофон закрывается. В неподдерживаемом браузере — тост «Режим разговора работает в Chrome или Edge».

☐ pass

Composer

T-CHAT-38 — Захват кадра камерой

- Mode/where: кнопка камеры в нижнем ряду.
- Steps:
 1. Нажми камеру, разреши доступ, сделай снимок.
- Expected: кадр прикрепляется как изображение (тот же путь, что загруженное фото); далее можно спросить «что на фото». Без камеры — тост «Камера недоступна на этом устройстве».

☐ pass

T-CHAT-39 — Прикрепление файлов и фото

- Mode/where: + → «Файлы и фото» (или drag-and-drop в поле).
- Steps:
 1. Через + → «Файлы и фото» выбери PDF/DOCX/CSV/изображение.
 2. Спроси «разбери и выпиши главное».
- Expected: текст документа извлекается и уходит в контекст; изображение идёт в vision-модель. Принимаются: image, .pdf, .docx, .txt, .csv, .md, .json, .xml, .html, .log.

☐ pass

T-CHAT-40 — Палитра команд по ⌘K

- Mode/where: горячая клавиша ⌘K (Ctrl+K).
- Steps:
 1. Нажми ⌘K — откроется палитра; набери «совет», запусти; ⌘K/Esc закрывают.
- Expected: модалка `CommandPalette` со списком/поиском команд; запуск команды применяет её; Esc закрывает.

☐ pass

T-CHAT-41 — Плюс-меню (режимы с примерами)

- Mode/where: кнопка `+` слева в нижнем ряду.
- Steps:
 1. Открой `+` — список пунктов с примерами «напр.: ...».
 2. Выбери «Супер-поиск · ответ с источниками [1] [2]».
- Expected: меню содержит реальные пункты (Файлы и фото, Поиск по файлам · база знаний (RAG), Умный поиск, Навыки, Маркет навыков, Запуск кода, Песочница API, Артефакты · Канвас, Картинка, Раздумья по шагам, Ресёрч, Супер-поиск, Браузер-в-чате, Оркестратор, Веб-поиск, Улучшить, Совет, Сравнить модели, Агент, Мои агенты, Без цензуры). Включённые помечены «вкл». Выбор сеет пример в плейсхолдер и применяет режим.

☐ pass

T-CHAT-42 — Песочница API (Playground) из плюс-меню

- Mode/where: `+` → «Песочница API · прогнать модель + сниппет».
- Steps:
 1. Открой `+` → «Песочница API».
- Expected: открывается модалка Playground — прогнать модель и забрать curl/fetch-сниппет; пункт помечается «вкл», пока открыт.

☐ pass

T-CHAT-43 — Улучшить промпт (разовая полировка)

- Mode/where: кнопка `Улучшить промпт` (Sparkles) в нижнем ряду.
- Steps:
 1. Введи сумбурный черновик, нажми «Улучшить промпт».
- Expected: спиннер на кнопке, затем текст в поле заменяется причёсанной формулировкой (POST `/api/improve`). Кнопка отключена при пустом поле / во время генерации.

☐ pass

T-CHAT-44 — Умный поиск (smart-RAG, мульти-запрос)

- Mode/where: `+` → «Умный поиск (мульти-запрос) · по памяти/докам».
- Steps:
 1. Включи «Умный поиск» (тост «Умный поиск вкл — несколько формулировок запроса...»).
 2. Спроси что-то, что есть в памяти/доках.
- Expected: при включённом — RAG-recall идёт мульти-запросом (MultiQuery + RRF), тост «🧠 вспомнил из памяти (умный поиск)»; при выключенном — обычный смысловой k-NN, тост «... (смысловой поиск)». Состояние сохраняется (`taiga.smartRag`).

☐ pass

Sessions

T-CHAT-45 — Новый чат

- Mode/where: создание нового чата (сайдбар / `/clear`).
- Steps:
 1. В существующем чате открой новый.
- Expected: история очищается, показывается пустое состояние (заголовок-морфинг, «Один интерфейс ко всему интеллекту планеты»), композер пуст.

☐ pass

T-CHAT-46 — Переключение между чатами (сайдбар)

- Mode/where: сайдбар со списком чатов.
- Steps:
 1. Создай 2 чата с разными сообщениями.
 2. Переключайся между ними в сайдбаре.
- Expected: каждый чат показывает свою историю; переключение не теряет сообщений.

☐ pass

T-CHAT-47 — Ветвление / дерево сообщений (branch)

- Mode/where: кнопка «ветка» (GitFork) под сообщением; переключатель «« 2/3 »».
- Steps:
 1. Под ответом нажми «ветка» (или отредактируй сообщение пользователя «править»).
 2. Сгенерируй альтернативный ответ.
- Expected: появляется переключатель веток `< index/total >`; кнопками `< >` переключаешься между вариантами ответа; оригинал цел.

☐ pass

T-CHAT-48 — «Заново» (regenerate)

- Mode/where: кнопка «заново» (RefreshCw) под ответом.
- Steps:
 1. Под ответом нажми «заново».
- Expected: генерируется новый ответ на тот же запрос; предыдущий доступен как ветка.

☐ pass

T-CHAT-49 — Сохранение и восстановление на перезагрузке

- Mode/where: F5 / перезагрузка вкладки.
- Steps:
 1. Веди диалог из нескольких сообщений.
 2. Перезагрузи страницу.
- Expected: открывается последний активный чат с полной историей (не пустой новый); дерево веток восстанавливается. (Исключение — «призрак»-чат: см. T-CHAT-53.)

☐ pass

T-CHAT-50 — Экспорт диалога/ответа

- Mode/where: кнопка «Экспорт» под ответом (ExportMenu) / `/export` / `/pdf`.
- Steps:
 1. Под ответом открой «Экспорт», выбери PDF / Word / Markdown.
- Expected: файл скачивается в выбранном формате; `/export` отдаёт весь диалог в Markdown, `/pdf` — последний ответ в PDF.

☐ pass

Onboarding

T-CHAT-51 — Пустое состояние + карточки онбординга

- Mode/where: первый запуск (или `localStorage.taiga.onboarded` сброшен), пустой чат.
- Steps:
 1. Очисти `taiga.onboarded` и открой `/app` (или новый чат при непройденном онбординге).
- Expected: блок «Что умеет Тайга» — 9 карточек по темам (Совет моделей, Сравнение, Мозг и Ресёрч, Картинки, Видео-Музыка-3D, Навыки и агенты, Память, МСР-коннекторы, Файлы и зрение) + чипы «Попробуй спросить — по режимам» (Чат/Мозг/Реле-Крафт/Совет/Сравнение/Ресёрч/Веб-Агент/Картинки). Тап по карточке/чипу ВПИСЫВАЕТ пример-промт в поле.

☐ pass

T-CHAT-52 — Запуск тура по возможностям + скрытие подсказок

- Mode/where: в онбординге кнопка «Пройти тур» / крестик «Скрыть подсказки».
- Steps:
 1. Нажми «Пройти тур» — открывается `CapabilityTour`.
 2. Закрой тур, затем нажми крестик «Скрыть подсказки».
- Expected: тур открывается (`capability-tour`); крестик помечает онбординг пройденным (`taiga.onboarded=1`), карточки больше не показываются; первое отправленное сообщение тоже завершает онбординг.

☐ pass

Edge cases

T-CHAT-53 — Чат-призрак (ничего не сохраняется)

- Mode/where: чип призрак (Ghost) / /ghost .
- Steps:
 1. Включи призрак , отправь сообщение, перезагрузи страницу.
- Expected: плейсхолдер поля «Призрак — этот чат нигде не сохранится...»; после перезагрузки диалог НЕ восстановлен (на диск ничего не пишется).

☐ pass

T-CHAT-54 — Пустая отправка отклоняется

- Mode/where: кнопка отправки при пустом поле.
- Steps:
 1. Оставь поле пустым (или только пробелы) и попробуй отправить (кнопка/Enter).
- Expected: кнопка ↗ неактивна (приглушена); отправка не происходит, пустого сообщения в истории нет.

☐ pass

T-CHAT-55 — Слишком много сообщений → 400 (>400)

- Mode/where: серверная валидация (SEC_MAX_MESSAGES=400).
- Steps:
 1. Сформируй запрос с историей >400 сообщений (длинный накопленный диалог) и отправь.
- Expected: бэкенд отвечает 400 с причиной «слишком много сообщений (>400)»; в пузыре появляется ⚠ Ошибка backend (400). (Окно памяти/ /compact помогают держаться ниже лимита.)

☐ pass

T-CHAT-56 — Слишком большой объем текста → 400 (>~4 МБ)

- Mode/where: серверная валидация (SEC_MAX_TOTAL_CHARS=4_000_000).
- Steps:
 1. Прикрепи/вставь суммарно >4 000 000 символов (огромный текст/файлы) и отправь.
- Expected: бэкенд возвращает 400 «слишком большой объем текста в сообщениях»; пузырь показывает ⚠ Ошибка backend (400).

☐ pass

T-CHAT-57 — Нет баланса (no-balance) у не-владельца

- Mode/where: не-owner аккаунт с нулевым/недостаточным балансом.
- Steps:
 1. Под не-owner с пустым балансом задай вопрос платной модели.
- Expected: ответ объясняет причину по-человечески (баланс/402 → «кончился баланс провайдера (пополни NanoGPT)» либо сообщение биллинга), без жжения денег; владелец видит баланс провайдеров отдельно (это не блокирует owner).

☐ pass

T-CHAT-58 — Очень длинный одиночный ввод (в пределах лимита)

- Mode/where: поле ввода.
- Steps:
 1. Вставь длинный (но <4 МБ суммарно) текст в одно сообщение и отправь.
- Expected: поле растёт до макс-высоты (~200px) со скроллом; отправка проходит, ответ стримится нормально; 400 не возникает.

☐ pass

Итого: 58 тестов · 9 групп (Modes / Streaming&Cost / Pre-send estimate / Commands / Agent / Voice / Composer / Sessions / Onboarding / Edge cases).

2 · Студия, медиа и воркфлоу

Тайга ИИ — QA:

Студия / Медиа /

Воркфлоу / Экспорты

/ Голос

Manual test catalog for the generation studio, cinema, playground, workflows, document exports, and voice/TTS. Run by hand at <http://localhost:3000/app> (backend :8777). The fixed account is "default" = OWNER — owner-only behaviour (free subscription models, hidden price) is called out per test.

How to reach each surface (verified):

- **Студия генерации** — режим-пилюля «Картинки» внизу у поля ввода (`mode-pills.tsx:11` , `chat.tsx:620` `renderImage` , mounted `chat.tsx:4459-4469` `MemoImageStudio`). The studio has a sub-bar with Картинки · Видео · Аудио · Аватар · Тулзы · Кино · Агенты · Музыка · 3D (`image-studio.tsx:59-69`).
- **Песочница (Playground)** — function-bar / command entry «Песочница API» (`chat.tsx:3126` , opens via `setPlaygroundOpen(true)` `chat.tsx:3176` , mounted `chat.tsx:5017`).
- **Воркфлоу** — Настройки → раздел «Воркфлоу» → кнопка открытия (`settings-panel.tsx:942-954` , `setWorkflowsOpen(true)` , mounted `settings-panel.tsx:1476`).
- **Голос / озвучка** — Настройки → «голос — озвучка и диктовка» (`settings-panel.tsx:860-925`).
- **Экспорт ответа** — меню « Экспорт» под каждым ответом Тайги (`chat.tsx:4656-4660` , `ExportMenu` `chat.tsx:5509+` : PDF · Word · Markdown).

Scope note: this plan covers **only what exists in code**. Notable absences verified and NOT tested as UI features: there is **no UI button that calls `exportPptx`** — `.pptx` / `toSlides` exist only as a library helper (`export-doc.ts:535` , `lib/artifacts/pptx.ts`), `grep exportPptx(` across all `.tsx` returns nothing. `.xlsx` / CSV export is reachable only through the **table artifact** (`artifacts/sheet-view.tsx`), not the answer export menu.

Image (генерация картинок)

T-STUDIO-01 — Basic image generation

- Where: Студия → вкладка «Картинки».
- Steps: Type a prompt (or tap a starter chip, e.g. «неоновый тигр в зимней тайге, кинематографично»), leave model on the default, press «Сгенерировать».
- Expected: Spinner placeholder in the canvas matching the aspect; on success one image appears in the result grid with a hover **Download** button; the gen is added to История. POSTs `/api/image` with `user:"default"` (`gen-image.ts:35`).

☐ pass (costs balance unless owner-free model)

T-STUDIO-02 — Price-before-gen shows ≈₽/≈\$ on the button

- Where: Студия → «Картинки».
- Steps: Pick a paid model; observe the «Сгенерировать» button label and the tier strip above it.
- Expected: Button shows `~$<gen_usd>` (approx marker `~`, `image-studio.tsx:443-447`). А **Платно** strip reads "Платно · модель спишет ~\$X за картинку" (`image-studio.tsx:465-466,1357-1370`). Model chips also show `~$X` (`image-studio.tsx:933,950`).

☐ pass

T-STUDIO-03 — Owner free subscription model shows (owner-only)

- Where: Студия → «Картинки», as OWNER.
- Steps: Select **Nano-GPT** free model (`ng:hidream` or `ng:qwen-image` , pinned to top for owner).
- Expected: Chip shows  ... \$0 ; tier strip turns green « **Бесплатно · модель входит в подписку (100 картинок/день)**»; button price reads `·  бесплатно` (`image-studio.tsx:442,463-464`). For a non-owner these models are not pinned and the path is paid.

☐ pass (owner-only)

T-STUDIO-04 — Negative prompt is a separate field (not concatenated)

- Where: Студия → «Картинки».
- Steps: Click «+ **негатив-промпт**» (`image-studio.tsx:1123`), type e.g. `размыто, лишние пальцы` , generate. Inspect the POST body in DevTools → Network → `/api/image` .
- Expected: Toggle reveals a textarea; request body carries a distinct `negative_prompt` field, NOT appended to `prompt` (`gen-image.ts:19-24` , only sent when non-empty). The negative content visibly suppresses those artifacts in the result.

☐ pass (costs balance)

T-STUDIO-05 — img2img from uploaded source + strength slider

- Where: Студия → «Картинки».
- Steps: Upload a source image («**загрузить картинку**»), confirm the hint "Загрузи фото — будем править/делать вариацию..." appears; a «**Сила правки · NN%**» slider shows (`image-studio.tsx:1239-1260`). Set strength low (≈20%), generate; then high (≈90%), generate.
- Expected: With a source present the run routes through `generateImageTool({tool:"edit", strength})` (`image-studio.tsx:726-727`); low strength stays close to original, high strength is a strong remake. Tier strip reads «**Платно · правка из фото спишет ~\$0.04**» (fixed price, model-independent, `image-studio.tsx:460`).

☐ pass (costs balance ~\$0.04)

T-STUDIO-06 — Seed reproducibility (lock seed)

- Where: Студия → «Картинки» → expand «**Продвинутые**» (`image-studio.tsx:1304-1311`).
- Steps: Generate once on auto seed; the panel shows "последний результат: seed N". Click « **закрепить N**», generate again with the same prompt/model.
- Expected: Locked seed produces a (near-)identical image; the header chip shows «**seed ** » and Seed field becomes "(зафиксирован)" (`image-studio.tsx:1311,1317-1322`). « **авто**» clears it back to random. Seed is read from the result `seed` field (`gen-image.ts:48` , `image-studio.tsx:746-747`).

☐ pass (costs balance ×2)

T-STUDIO-07 — Aspect ratios map to real pixel dimensions

- Where: Студия → «Картинки» → «**Формат**» chips.
- Steps: Cycle through 1:1, 16:9, 9:16, 4:3, 3:4 and generate one each (or just observe canvas frame).
- Expected: Five chips exist (`image-studio.tsx:122`); canvas frame matches the chosen aspect; request `width / height` follow `ASPECT_DIM` (e.g. 16:9 → 1344×768, 9:16 → 768×1344, `image-studio.tsx:132-138,723`), not a fixed 1024².

☐ pass (costs balance)

T-STUDIO-08 — Advanced ParamForm «Доп. параметры»

- Where: Студия → «Картинки» → «Продвинутые» → «**Доп. параметры**» (`image-studio.tsx:1345-1351`).
- Steps: Set **Сэмплер** (enum: `euler/euler_a/dpmpp_2m/...`), **Расписание** (normal/karras/...), **CLIP skip** (slider 1–4). Generate; inspect `/api/image` body.
- Expected: Three schema-driven controls render via `ParamForm` (`param-form.tsx` : enum→select, integer min&max→slider). Values other than «по умолчанию»/empty are spread into the request body (`cleanAdvParams` `image-studio.tsx:176-184` , `spread ...extra` `image-studio.tsx:742`). Leaving all at default sends none of them.

☐ pass (costs balance)

T-STUDIO-09 — Multi-count (1 / 2 / 4)

- Where: Студия → «Картинки» → «**Сколько**» chips.
- Steps: Choose 4, generate (no source image — count only applies to from-scratch path).
- Expected: Chips 1/2/4 (`image-studio.tsx:1294`); canvas shows 4 loading tiles in a 2-col grid, then 4 results; 4 parallel `/api/image` calls via `Promise.all` (`image-studio.tsx:728-745`). With an `img2img` source uploaded, count is ignored (single edit call).

☐ pass (costs balance ×count)

T-STUDIO-10 — Steps & CFG sliders

- Where: Студия → «Картинки» → «Продвинутые».
- Steps: Drag «Шаги» (0–50) and «Сила промпта CFG» (0–20, step 0.5), generate.
- Expected: Label shows live value or "по умолчанию" at 0 (`image-studio.tsx:1334,1338`); 0 omits the field (`steps || undefined, cfg || undefined, image-studio.tsx:738–739`); non-zero values reach the request.

☐ pass (costs balance)

T-STUDIO-11 — History persistence & re-load

- Where: Студия → «Картинки» canvas.
- Steps: Generate a few images, reload the page, return to «Картинки», click a history thumbnail.
- Expected: «История генераций» grid shows past gens; only `http(s)` URLs persist to `localStorage` (`taiga.gens` , base64 stripped, capped 40, `image-studio.tsx:497–508`). Clicking a thumb restores its prompt and shows it as the current result (`image-studio.tsx:1515–1521`).

☐ pass

T-STUDIO-12 — Free code-render toggle (diagram/poster/SVG)

- Where: Студия → «Картинки».
- Steps: Enable « Код-рендер · диаграмма / постер / график / SVG» toggle (`image-studio.tsx:1063–1094`), enter e.g. «блок-схема процесса оплаты», generate.
- Expected: Tier strip turns green «**Бесплатно · рендер кодом, платная модель не вызывается**»; button reads « бесплатно (код)»; result model tag is « Код-рендер» (`image-studio.tsx:703`). Also: a prompt that "looks like a diagram" surfaces an inline suggestion link (`image-studio.tsx:1105–1113`).

☐ pass

Image errors (UX категорий провала)

All image failures funnel through `fail()` → `classifyFail()` which buckets into **provider / balance / refused / generic** by lib text or explicit `kind` (`image-studio.tsx:206–274,517–534`). The canvas shows a category icon + `ErrorState` (title + «попробовать снова») and never a stuck spinner.

T-STUDIO-13 — Provider-down message + retry

- Where: Студия → «Картинки».
- Steps: Trigger a provider/network failure (stop backend :8777, or force a 502/timeout), generate.
- Expected: Canvas shows the **CloudOff** sky-tinted icon, title «**Провайдер недоступен**», copy "Сервис генерации сейчас не отвечает. Это не твоя ошибка...", and a retry control; left button switches to «**Повторить**» with price (`image-studio.tsx:1401-1403,1464-1494`). No infinite spinner.

☐ pass

T-STUDIO-14 — No-balance message + retry

- Where: Студия → «Картинки».
- Steps: With insufficient balance (or backend returning "недостаточно средств" / "insufficient"), generate a paid model.
- Expected: **CreditCard** amber icon, title «**Недостаточно баланса**», copy "На балансе не хватает... Пополни счёт или выбери модель подешевле — и жми «Повторить»." (`image-studio.tsx:249-255,1460-1462`).

☐ pass

T-STUDIO-15 — Refused / moderation message + retry

- Where: Студия → «Картинки».
- Steps: Submit a prompt the provider filter rejects (returns "отклонён"/"policy"/"nsfw"/"moderation").
- Expected: **ShieldAlert** fuchsia icon, title «**Запрос отклонён**», copy about reformulating softer (`image-studio.tsx:256-264`).

☐ pass

T-STUDIO-16 — Refund chip «средства возвращены»

- Where: Студия → «Картинки».
- Steps: Trigger a paid failure where the backend returns `refunded:true` (or `refund:>0`).
- Expected: Below the error card a green chip « **средства возвращены — списания за неудачную генерацию нет**» (`wasRefunded image-studio.tsx:278-284` , render 1495-1500). Absent when no refund flag.

☐ pass

T-STUDIO-17 — Cancel (stop) is not an error

- Where: Студия → «Картинки» / Видео.
- Steps: Start a generation, press «**стоп**» / «**Отменить**» mid-run.
- Expected: Returns to a clean canvas, no error card, no refund chip ("отменено" branch, `image-studio.tsx:521-526,766-769`).

☐ pass

T-STUDIO-18 — Retry from error re-runs the same request

- Where: Студия → «Картинки».
- Steps: After any error card, click the retry / «**Повторить**» button.
- Expected: `generate()` re-runs with the same inputs; previous error/refund state is cleared first (`setLastError("")`, `setRefunded(false)` `image-studio.tsx:539-540`); button price label persists.

☐ pass (costs balance)

Video / Cinema (Видео + Кино)

T-STUDIO-19 — Text-to-video clip generation

- Where: Студия → вкладка «**Видео**».
- Steps: Pick a video model (sortable **Топ** / **Дешевле** / **Дороже** / **А-Я**, `image-studio.tsx:881-886`), describe the clip, choose **Длительность** 5с/8с (`image-studio.tsx:1283`) and **Формат**, press generate.
- Expected: Submit→poll flow; canvas shows "генерится... Nc" with provider status and an «**Отменить**» button (`image-studio.tsx:1426-1438`); button price is exact `$<usd>` (no `~`, `image-studio.tsx:422-423`). Result is an autoplaying `<video controls>` (`image-studio.tsx:1664-1665`). "видео делается ~30-120 сек" hint shows.

☐ pass (costs balance)

T-STUDIO-20 — Image-to-video start frame (i2v) & r2v reference

- Where: Студия → «Видео».
- Steps: For an i2v model upload a «**Старт-кадр**»; pick an **r2v** model and confirm a «**Референс-видео (обяз.)**» uploader appears with the "\$1.95, грузи файлом" hint (`image-studio.tsx:1167-1171`).
- Expected: Start frame is sent as `imageUrl` ; r2v requires a video file or blocks with "Загрузи референс-видео (файлом)" (`image-studio.tsx:588`).

☐ pass (costs balance)

T-STUDIO-21 — Avatar (talking head): face + audio ≤30s guard

- Where: Студия → вкладка «**Аватар**».
- Steps: Upload a face photo (обяз.) and a voice audio (обяз.); try an audio > 30s.
- Expected: Both inputs required (`image-studio.tsx:589-590`); audio longer than ~30.5s is blocked client-side with "Аудио Nc — для аватара нужно ≤30с, обрежь" (`image-studio.tsx:592-601`) before any paid call. The ⚠ "аудио до 30 секунд" hint is shown (`image-studio.tsx:1141`).

☐ pass (costs balance)

T-STUDIO-22 — Video tools (Тулзы) input requirements

- Where: Студия → вкладка «**Тулзы**» → «Видео-тулзы».
- Steps: Pick upscale/extend/edit (needs video only) and a face-swap tool (needs video + face). Observe the dynamic hint (`T00L_HINT` , `image-studio.tsx:84-90,1171`).
- Expected: Video upload required ("Загрузи видео", `image-studio.tsx:602`); face-swap also requires a face photo ("Загрузи фото лица", `603`). The «Видео-тулзы / Фото-тулзы» switch is at the top (`image-studio.tsx:814-835`).

☐ pass (costs balance)

T-STUDIO-23 — Video error category + retry (no stuck spinner)

- Where: Студия → «Видео».
- Steps: Force a failure (provider down / refused / no balance).
- Expected: Same four-category classification as images, canvas error card with icon + retry; progress poller cleared (`setVidProgress(null)`), button flips to «Повторить» (`image-studio.tsx:626-628` , classify shared `206-274`).

☐ pass

T-STUDIO-24 — Cinema: build a storyboard, per-scene kind & model

- Where: Студия → вкладка «Кино» (`CinemaStudio`).
- Steps: Add scenes («+ Сцена»), per scene pick a kind tab **видео / картинка / оживить / аватар** (`cinema-studio.tsx:116-121`), pick a model from the per-scene `<select>` (default "авто (...)").
- Expected: Each scene is an independent card; model list is filtered by kind (t2v/i2v/avatar/image, `cinema-studio.tsx:172-179`); header counter shows "N сцен · M готово" (`cinema-studio.tsx:330-331`).

☐ pass

T-STUDIO-25 — Cinema: per-scene pro-camera + «Доп. параметры»

- Where: Кино → a video/animate scene.
- Steps: Set **план / движение / объектив / диафрагма** dropdowns (`cinema-studio.tsx:123-148, 461-475`); expand «Доп. параметры» and set Длительность (5/8/10s) / fps (8-30) / Разрешение (480p/720p/1080p) (`VIDEO_ADV_PARAMS` `cinema-studio.tsx:70-101`).
- Expected: Camera selectors append phrases to the prompt (`cam join`, `cinema-studio.tsx:204-205`); adv params spread into `/api/video` body via `cleanSceneParams` , default/empty omitted (`cinema-studio.tsx:105-114, 209`). Image scenes hide the adv params block (`481`).

☐ pass

T-STUDIO-26 — Cinema: shoot one / shoot all / play / export MP4

- Where: Кино.
- Steps: «Снять» a single scene, then «Снять всё»; «Проиграть» the storyboard; «Скачать MP4».
- Expected: Single/all shoot run scenes; «Проиграть» opens a full-screen player that auto-advances (image scenes after 3.5s, video on `onEnded`, `cinema-studio.tsx:309–321,565–579`); export concatenates done scenes via `exportCinema` and downloads `taiga-film.mp4` (`cinema-studio.tsx:286–302`). Buttons disable when nothing is ready.

☐ pass (costs balance)

T-STUDIO-27 — Cinema: Режиссёр (auto storyboard from one idea)

- Where: Кино → fuchsia «Режиссёр» bar.
- Steps: Type a one-line film idea, press «Сценарий по идее» (or Enter).
- Expected: Button shows "пишу сценарий...", then scenes are auto-populated from `planStoryboard` with kinds/prompts/camera mapped (`cinema-studio.tsx:259–284`); empty plan → toast "Режиссёр не справился...". (Reaching Кино via Агенты-галерея passes `initialIdea` and auto-directs once, `cinema-studio.tsx:165–169`.)

☐ pass

T-STUDIO-28 — Cinema: per-scene error category + «переснять» + refund

- Where: Кино.
- Steps: Force a scene failure.
- Expected: Scene card thumbnail swaps to a category icon (CreditCard/ShieldAlert/CloudOff/AlertTriangle, `cinema-studio.tsx:394–413`); an inline strip shows the category title + reason + «переснять» button (`classifyScene` `cinema-studio.tsx:47–57,504–543`); a « средства возвращены» chip appears if `refunded` (`537–541`). Cancel ("отменено") quietly resets the scene (`239`).

☐ pass

Playground (Песочница)

T-STUDIO-29 — Pick a text model

- Where: Песочница (modal).
- Steps: Open the model `<select>` ; pick a model.
- Expected: Only text models listed (image/video filtered out), sorted by `smart desc` (`playground.tsx:165–170`); options show name + `· free` + ctx where present (`playground.tsx:374–378`). Header shows "N моделей".

☐ pass

T-STUDIO-30 — Form mode: system prompt + sampling params + run (streamed)

- Where: Песочница → «Форма» tab.
- Steps: Fill **Системный промпт**, **Запрос**, adjust Макс. токенов / Температура / Top-p (`FORM_SCHEMA playground.tsx:56–87`), press «Запустить».
- Expected: Result panel streams tokens with a blinking cursor (SSE `data:{type:"delta",text}` , `playground.tsx:301–307,500–503`); empty input blocks with "Напиши, что спросить у модели" (`playground.tsx:268`). «Остановить» aborts (`playground.tsx:254–258,453–460`).

☐ pass (costs balance unless owner/free)

T-STUDIO-31 — JSON mode toggle + validation

- Where: Песочница → «JSON» tab.
- Steps: Switch to JSON; the editor is pre-filled from the form body (`playground.tsx:220–225`). Break the JSON, then fix it.
- Expected: Invalid JSON shows "⚠ " (`playground.tsx:447`) and Run errors with "JSON-тело запроса невалидно..." (`playground.tsx:264–266`). Valid JSON is sent verbatim as the body.

☐ pass

T-STUDIO-32 — Live price on Run button (hidden for free/owner)

- Where: Песочница.
- Steps: As OWNER, observe the button. Then (non-owner) pick a paid model and a free model.
- Expected: Price chip uses `approxRub(priceUsd)` and is shown only when `!isOwner && !model.free && priceUsd>0` (`playground.tsx:235-236,468-477`). Free model shows green « бесплатно»; owner sees no price chip; a sub-line "оценка по тарифу модели · спишется по факту" shows only when priced (`playground.tsx:480-484`).

☐ pass

T-STUDIO-33 — Copy API/embed snippet (curl & fetch, \$TAIGA_TOKEN, no secrets)

- Where: Песочница → «API-сниппет» footer.
- Steps: Toggle **curl / fetch**, press «**копировать**», paste elsewhere.
- Expected: curl snippet uses `-H "Authorization: Bearer $TAIGA_TOKEN"` (`playground.tsx:123`); fetch uses `Bearer ${process.env.TAIGA_TOKEN}` (`playground.tsx:135`); both target `https://taiga.chat/api/chat` and contain **no real keys**. Button confirms with «**скопировано**» + check for ~1.4s (`playground.tsx:244-252,533-534`).

☐ pass

T-STUDIO-34 — Backend / model error surfacing

- Where: Песочница.
- Steps: Run against a backend that returns non-OK, or trigger an SSE `error` event.
- Expected: Red error box "Бэкенд · ..." or the model error message; running flag cleared (`playground.tsx:286-291,307,495-498`); no stuck spinner.

☐ pass

Workflows (Воркфлоу)

T-STUDIO-35 — Open gallery, see the 4 backend templates

- Where: Настройки → «Воркфлоу».
- Steps: Open Воркфлоу; stay on the «Шаблоны» tab.
- Expected: GET `/api/workflow` returns 4 templates (`server.py:1810–1862`): «Ресёрч-бриф» (research-brief, web→chat), «Картинка из идеи» (image-from-idea, chat→image), «Вопрос по документам» (doc-qa, rag→chat), «Переписать и отполировать» (rewrite-polish, single chat step). Each card shows title, desc, step chips with arrows, and "N шаг/шага/шагов" (`workflows.tsx:312–371`). Tabs «Мои» / «Опубликованные» show "скоро" empty states (`workflows.tsx:249–263`).

☐ pass

T-STUDIO-36 — Run «Переписать и отполировать» (rewrite-polish) → step output + final result

- Where: Воркфлоу → click the rewrite-polish card.
- Steps: In RunView enter rough text (e.g. a messy paragraph), press «Запустить» (or ⌘/Ctrl+Enter).
- Expected: POST `/api/workflow {user:"default", template_id:"rewrite-polish", input}` (`workflows.tsx:387–395`). Button shows "Запускаю..."; a numbered step card «Полировка текста» appears with its output and a green check (`workflows.tsx:486–508`); the green «Готово» final result block shows the polished text (`workflows.tsx:511–518`). Verified template id `rewrite-polish` is a single chat step (`server.py:1851–1861` ; test `tests/test_endpoints.py:270–275`).

☐ pass (costs balance unless owner/free)

T-STUDIO-37 — Run a multi-step template (research-brief / doc-qa)

- Where: Воркфлоу.
- Steps: Run «Ресёрч-бриф» with a topic.
- Expected: Two step cards in order — «Поиск в вебе» then «Сводка-бриф» — each with its output, then the final result (`server.py:1815–1822` steps; `{steps.0}` feeds step 2). doc-qa similarly runs «Поиск по базе» → «Ответ по контексту».

☐ pass (costs balance)

T-STUDIO-38 — Workflow error handling (ok:false)

- Where: Воркфлоу → RunView.
- Steps: Force a failure (backend returns `{ok:false, error}` or non-2xx).
- Expected: A red «Не получилось» card shows `run.error`; no final result block; running flag cleared (`workflows.tsx:396–407,479–484`). Empty input keeps the button disabled (`workflows.tsx:457`).

☐ pass

T-STUDIO-39 — Templates load error + retry

- Where: Воркфлоу → «Шаблоны».
- Steps: Open with backend down, then restore and retry.
- Expected: `ErrorState` "Не удалось загрузить" with retry that re-fetches `/api/workflow` (`workflows.tsx:222–229`); a successful template list replaces it.

☐ pass

Exports (выгрузка документов)

T-STUDIO-40 — Answer → .docx (real Word, opens cleanly)

- Where: Chat → any Тайга answer → « Экспорт» → **Word (.docx)** (`chat.tsx:5557`).
- Steps: Ask for a structured answer (headings, **bold**, a list, a table, a code block); export Word; open in Word/Pages/LibreOffice.
- Expected: Toast "Скачиваю Word"; file is genuine OOXML built with the `docx` lib via `Packer.toBlob` (`export-doc.ts:476–527`), NOT the old HTML fake. Headings map to `HeadingLevels`, bold/italic/code/strike runs, hyperlinks, bulleted & numbered lists, blockquotes, code blocks (mono + shading), and tables (bordered, shaded header) all render and open without repair prompts (`export-doc.ts:204–468`). Reads the rendered message node (`data-export-id`, `chat.tsx:4600–4602,2407–2416`).

☐ pass

T-STUDIO-41 — Answer → PDF (multipage)

- Where: Chat answer → « Экспорт» → **PDF (.pdf)** (`chat.tsx:5556`).
- Steps: Export a long answer; observe toast and file.
- Expected: "Готовлю PDF..." then "Скачиваю PDF"; light A4 sheet with readable typography, code blocks light-themed, tables/images constrained; content longer than one page is sliced across multiple pages (`export-doc.ts:39-165,2426-2429`). A spinner shows on the menu while rendering (`exportBusyId`).

☐ pass

T-STUDIO-42 — Answer → Markdown (.md)

- Where: Chat answer → « Экспорт» → **Markdown (.md)** (`chat.tsx:5558`).
- Steps: Export; open the file.
- Expected: Toast "Скачиваю Markdown"; raw answer text saved as `text/markdown` with a safe, title-derived filename (`export-doc.ts:29-32,2403-2404` , `titleFromText`).

☐ pass

T-STUDIO-43 — Table artifact → .xlsx (real Excel) and CSV

- Where: Chat answer that contains a markdown table or `""csv` block → the inline **Таблица** artifact (`artifacts/sheet-view.tsx`) — OR a table under markdown (`markdown.tsx:128`).
- Steps: Click «**Excel**» and «**CSV**» under the table; open the .xlsx in Excel.
- Expected: Excel button builds a real `.xlsx` via SheetJS (`exportXlsx` from `artifacts/sheet` , re-exported `export-doc.ts:542`); CSV downloads plain CSV; per-button spinner while busy (`sheet-view.tsx:33-37,109-134`). Opens cleanly as a spreadsheet with columns/rows intact.

☐ pass

T-STUDIO-44 — Artifact downloads: chart PNG/SVG, diagram SVG/PNG, 3D PNG, web .html, anim .html

- Where: Chat artifacts (chart / mermaid / three / web-preview / anim views).
- Steps: For each artifact use its download control.
- Expected: Chart → `taiga-chart.png` / `.svg` (`chart-view.tsx:44,50`); Mermaid diagram → SVG/PNG (`mermaid-view.tsx:86-93`); 3D → frame `taiga-3d.png` (`three-view.tsx:111`); web preview → `.html` (`web-preview.tsx:88`); anim → `taiga-anim.html` (`anim-view.tsx:31`).

☐ pass

Note: No `.pptx` export exists in the UI (helper only). Do not test a "Скачать PowerPoint" button — there isn't one (verified: zero `exportPptx()` call sites in `.tsx`).

Voice / TTS (голос и озвучка)

T-STUDIO-45 — Studio TTS («Аудио»): generate speech, free, no balance

- Where: Студия → вкладка «Аудио».
- Steps: Type text, press «Сгенерировать».
- Expected: Single voice «Русский · бесплатно» (`image-studio.tsx:95-97`); tier strip green «Бесплатно · озвучка не списывает баланс» (`image-studio.tsx:452-453`); result is an autoplaysing `<audio controls>` card (`image-studio.tsx:1658-1663`); routes to `/api/free-tts lang ru` (`audio-gen.ts:37-46`). Empty text blocks with "Впиши текст для озвучки" (`image-studio.tsx:544`).

☐ pass

T-STUDIO-46 — Voice provider picker (free vs paid) in Settings

- Where: Настройки → «Голос — озвучка и диктовка».
- Steps: Toggle **Озвучка (TTS)** between «Бесплатно (Google)» and «Платно (лучше качество)» (`settings-panel.tsx:888-906`).
- Expected: Choice persists to `taiga.voice.ttsProvider` (`voice-pref.ts, saveVoicePref`); free → `/api/free-tts ($0)`, `nanogpt` → `/api/audio` with a soft fallback to free if the paid call fails (`audio-gen.ts:22-35`).

☐ pass (paid path costs balance)

T-STUDIO-47 — Per-answer «озвучить» button (manual TTS replay)

- Where: Chat → message actions under a Тайга answer.
- Steps: Click the «озвучить» (Volume2) action.
- Expected: Icon → spinner while loading → VolumeX/«стоп» while playing; calls `speakMessage` → `generateAudio` (text capped 4000 chars) (`chat.tsx:4643-4655, 2451-2466`). "Озвучка недоступна — попробуй позже" toast on failure; clicking again stops playback.

☐ pass

T-STUDIO-48 — Auto-speak replies

- Where: Настройки → «Голос» → «**Озвучивать ответы вслух**» toggle (`settings-panel.tsx:864-882`).
- Steps: Enable autoSpeak; send a chat message; wait for the reply to finish.
- Expected: Each new completed answer is auto-spoken once (de-duped via `autoSpokenRef` , `chat.tsx:1027-1039`); reflects the chosen TTS provider; disabling stops further auto-speech. Changing the setting in another tab is picked up live (`chat.tsx:1010-1011`).

☐ pass

T-STUDIO-49 — STT input language picker

- Where: Настройки → «Голос» → «**Язык голосового ввода**» select (`settings-panel.tsx:919-925`).
- Steps: Change the language (e.g. ru-RU → en-US); use voice dictation.
- Expected: 8 languages available (`voice-pref.ts:STT_LANGS`); choice persists (`taiga.voice.sttLang`) and is used by the Web Speech dictation.

☐ pass

Photo tools (Фото-тулзы)

T-STUDIO-50 — Upscale 2x/4x

- Where: Студия → «Тулзы» → «**Фото-тулзы**» → **Апскейл** (\$0.02).
- Steps: Upload a photo, pick 2x or 4x (`photo-tools.tsx:178-199`), press «**Применить** · ~\$0.02».
- Expected: Routes `generateImageTool({tool:"upscale", scale})` (`photo-tools.tsx:62-64`); result image with Download; «**Готово ✓**». Missing photo blocks with "Загрузи фото".

☐ pass (costs balance ~\$0.02)

T-STUDIO-51 — Edit (Переделать) by prompt

- Where: «Фото-тулзы» → **Переделать** (\$0.04).
- Steps: Upload, describe the change (e.g. "сделай фон закатным, добавь снег"), apply.
- Expected: Requires a prompt ("Опиши, что изменить", `photo-tools.tsx:55`); routes tool `edit` ; returns edited image.

☐ pass (costs balance ~\$0.04)

T-STUDIO-52 — Style presets

- Where: «Фото-тулзы» → **Стиль** (\$0.04).
- Steps: Upload, pick a style chip (Аниме / Ghibli / Масло / Акварель / Киберпанк / Нуар / 3D-рендер / Комикс, `photo-tools.tsx:14–23`), apply.
- Expected: Maps to an `edit` call with the preset prompt (`photo-tools.tsx:62–63`); restyled image returned.

☐ pass (costs balance ~\$0.04)

T-STUDIO-53 — Photo-tools error + «Повторить»

- Where: «Фото-тулзы».
- Steps: Force a failure.
- Expected: Result pane shows a clean **«Не вышло»** card with the message and a **«Повторить»** button (`photo-tools.tsx:233–249`); no raw error, no stuck spinner.

☐ pass

Music & 3D (доп. режимы студии)

T-STUDIO-54 — Music generation (Музыка) + lyrics

- Where: Студия → вкладка **«Музыка»**.
- Steps: Pick a music model (MiniMax / Stable Audio / Lyria, sorted featured-first, `image-studio.tsx:405–415,986–1015`), describe style/mood/tempo, optionally add **«Текст песни»** for vocals (`image-studio.tsx:1018–1029`), generate.
- Expected: Submit→poll with "музыка ~30-60 сек"; result is an audio card; tier strip **«Платно · трек спишет ~\$X»** (`image-studio.tsx:472–474`). If no music models load, shows "Музыкальный провайдер сейчас недоступен" (`image-studio.tsx:989–992`).

☐ pass (costs balance)

T-STUDIO-55 — 3D режим is paused (provider maintenance)

- Where: Студия → вкладка «3D» (sub-bar shows a «пауза» badge, `image-studio.tsx:1615-1622`).
- Steps: Open the 3D tab; try to generate.
- Expected: An amber banner «3D из фото — временно недоступно» (`image-studio.tsx:862-872`); the generate button is replaced by a disabled «3D временно недоступно» (`image-studio.tsx:1373-1380`); attempting it only toasts the maintenance message — no hung request (`image-studio.tsx:662-664`). (`TD3_AVAILABLE = false` , `image-studio.tsx:57` .)

☐ pass

3 · Настройки, UI и UX

Тайга ИИ — QA:

Настройки / UI / UX

(ручной прогон)

App: `http://localhost:3000/app` · backend :8777 ·
аккаунт `default` = ВЛАДЕЛЕЦ (OWNER).

Важно про owner-гейт. В `chat.tsx` панель монтируется с `isOwner` (жёстко `true`, строка 4758) и `user="default"`. Значит на этом аккаунте все три owner-секции («личность Тайги», «Подписка — лимиты», «Диагностика и здоровье») ВИДНЫ. Чтобы проверить, что они скрыты у не-владельца, нужен аккаунт с `billing.owner=false` на бэке — для большинства owner-панелей (sub-meters / diagnostics) гейт ещё и серверный (компонент рендерит `null` при `owner=false`), поэтому отдельно отмечено, где гейт клиентский (пункт в списке настроек) и где серверный (содержимое панели).

Как открыть настройки. Левый сайдбар → кнопка «Настройки» (иконка Settings2). Панель — модалка `role="dialog" aria-modal="true"`, заголовок «Настройки», подпись «для аккаунта default».

Реальные группы навигации (порядок в левом меню, файл `settings-panel.tsx:74`):

1. Профиль и личность · 2. Модели и режимы · 3. Память и знания · 4. Студия и медиа · 5. Использование и подписка · 6. Интеграции (MCP / BYOK) · 7. Оформление · 8. Разное · Что нового.

Обозначения: `(owner-only)` — секция/панель видна/работает только у владельца.

Nav&Search

T-SET-01 — Открытие и базовый каркас настроек

- Where: сайдбар → «Настройки».
- Steps: открой панель; осмотри шапку.
- Expected: модалка по центру (десктоп) с заголовком «Настройки» и подписью «для аккаунта default»; есть поле поиска «Найти настройку...»; слева вертикальное меню из 8 групп; справа содержимое активной группы (по умолчанию «Профиль и личность»); крестик закрытия справа сверху.

☐ pass

T-SET-02 — Переключение групп навигации

- Where: левое меню настроек.
- Steps: кликай по группам по очереди: Профиль и личность → Модели и режимы → Память и знания → Студия и медиа → Использование и подписка → Интеграции (MCP / ВУОК) → Оформление → Разное · Что нового.
- Expected: активная группа подсвечена (фиолетовый фон, фиолетовая иконка, `aria-current`); справа меняется набор блоков; контент скроллится независимо.

☐ pass

T-SET-03 — Поиск «память» фильтрует по всем группам

- Where: поле «Найти настройку...».
- Steps: введи `память`.
- Expected: левое меню групп скрывается; показываются все совпавшие блоки из разных групп (напр. «что Тайга знает о тебе», «память — как ищем по прошлым чатам», «авто-память», «Память: бюджет контекста» относится к dev-панели). Сверху появляется счётчик «Найдено: N».

☐ pass

T-SET-04 — Поиск «тема» прыгает к оформлению

- Where: поле поиска.
- Steps: введи `тема`.
- Expected: остаётся блок «Тема и акцент» (группа Оформление). Счётчик «Найдено: 1» (или больше, если совпадут ключевые слова).

☐ pass

T-SET-05 — Поиск без совпадений

- Where: поле поиска.
- Steps: введи бессмыслицу, напр. `zzzqqq`.
- Expected: текст «По запросу ничего не нашлось — попробуй другое слово.»; счётчик показывает «Ничего не найдено».

☐ pass

T-SET-06 — Очистка поиска возвращает навигацию

- Where: поле поиска.
- Steps: введи запрос, затем нажми крестик «Очистить поиск» (или сотри текст).
- Expected: поле очищается; левое меню групп снова показывается; контент возвращается к активной группе.

☐ pass

T-SET-07 — Клавиатурная навигация по группам (roving focus)

- Where: левое меню, фокус на пункте группы.
- Steps: поставь фокус (Tab) на пункт группы; жми ↓/↑ (или →/←), затем Home и End.
- Expected: фокус и активная группа перемещаются по непустым группам циклически; Home — первая, End — последняя.

☐ pass

T-SET-08 — Esc закрывает, фокус возвращается на триггер

- Where: открытая панель.
- Steps: нажми Esc.
- Expected: панель закрывается; фокус возвращается на кнопку «Настройки» в сайдбаре.

☐ pass

T-SET-09 — Клик по затемнённому фону закрывает

- Where: открытая панель.
- Steps: кликни по тёмному фону вне карточки.
- Expected: панель закрывается. Клик внутри карточки НЕ закрывает.

☐ pass

T-SET-10 — Слип-линк «К содержимому настроек»

- Where: открытая панель, начало таб-обхода.
- Steps: открой панель и жми Tab от начала.
- Expected: первым фокусируемым элементом всплывает видимая ссылка «К содержимому настроек» (фиолетовая, в левом верхнем углу); по Enter фокус прыгает в область контента.

☐ pass

Profile&Persona

T-SET-11 — Системный промпт (мастер-промпт) — переход

- Where: Профиль и личность → блок «системный промпт».
- Steps: кликни «Мастер-промпт и ядро памяти...».
- Expected: панель настроек закрывается и открывается мастер-панель (через `onOpenMaster`).

☐ pass

T-SET-12 — Личность Тайги на сервере: редактирование и сохранение (owner-only)

- Where: Профиль и личность → «личность Тайги (на сервере)».
- Steps: дождись загрузки текстового поля (placeholder = дефолтная личность); впиши свой текст; убери фокус (blur).
- Expected: при blur уходит POST `/api/identity` ; справа появляется «Сохранено на сервере» (зелёным с галкой), затем гаснет.

☐ pass · (owner-only)

T-SET-13 — Личность persists после перезагрузки (owner-only)

- Where: та же секция.
- Steps: сохрани текст, закрой панель, перезагрузи страницу, снова открой секцию.
- Expected: GET `/api/identity` подтягивает сохранённый текст в поле (значение сохранилось на сервере, не в localStorage).

☐ pass · (owner-only)

T-SET-14 — «вернуть дефолт» личности (owner-only)

- Where: та же секция.
- Steps: нажми «вернуть дефолт» (иконка RotateCcw).
- Expected: поле очищается, уходит POST с пустой персоной; бэкэнд возвращает дефолтную личность; сообщение «Сохранено на сервере».

☐ pass · (owner-only)

T-SET-15 — Темперамент агента (Осторожный / Автономный)

- Where: Профиль и личность → «решает сам — насколько агент самостоятелен».
- Steps: переключи между «Осторожный» и «Автономный — решает сам».
- Expected: выбранный вариант подсвечен (фиолетовая рамка + галка справа); выбор применяется сразу (`onTemperament`).

☐ pass

T-SET-16 — Кастомные инструкции: два поля и применение к ответам

- Where: Профиль и личность → «кастомные инструкции» → открой панель.
- Steps: заполни «Что Тайге знать о тебе» (напр. «Меня зовут Дамир») и «Как Тайге отвечать» (напр. «Обращайся на ты, коротко»); нажми «Сохранить» (или закрой — авто-сохранение при закрытии, если есть несохранённые правки).
- Expected: статус в футере переключается «Не сохранено» → «Сохранено» / «Всё сохранено»; оба поля склеиваются в мастер-промпт (`taiga.master.v1`). Затем задай в чате вопрос — ответ учитывает обе инструкции.

☐ pass

T-SET-17 — Кастомные инструкции переживают переоткрытие

- Where: панель кастомных инструкций.
- Steps: сохрани, закрой, переоткрой панель.
- Expected: оба поля восстановлены из мастер-промпта (`parse()`) разбирает блоки «### Что Тайге знать обо мне» / «### Как Тайге отвечать»).

☐ pass

Memory

T-SET-18 — Библиотека промптов / Память / Закладки открываются

- Where: Память и знания → «Библиотека промптов», «Что Тайга знает о тебе», «Закладки».
- Steps: открой каждую панель по кнопке.
- Expected: каждая открывается своей модалкой; контент грузится (ленивый чанк, спиннер при первой загрузке).

☐ pass

T-SET-19 — Режим памяти: Локально / Сервер

- Where: Память и знания → «память — как ищем по прошлым чатам».
- Steps: переключи «Локально — приватно» ↔ «Сервер — умный поиск по смыслу».
- Expected: выбранный вариант подсвечен изумрудной рамкой + галка; применяется сразу (`onMemMode`).

☐ pass

T-SET-20 — Авто-память (тумблер)

- Where: Память и знания → «авто-память».
- Steps: переключи тумблер «Запоминать факты автоматически».
- Expected: тумблер меняет цвет (циан, когда включён); состояние сохраняется (`onMemAuto`).

☐ pass

T-SET-21 — Память: бюджет контекста — `protected_recent 2–40 (dev-mode)`

- Where: Модели и режимы → «режим разработчика — настройки на каждый режим» → включи тумблер «Включить ручки для опытных» → «Настройки на каждый режим» → секция «Память: бюджет контекста».
- Steps: двигай слайдер/число «защищать последние N сообщений».
- Expected: значение зажимается в диапазон 2–40 (нельзя выйти за края); подпись «N сообщ.» обновляется live.

☐ pass

T-SET-22 — Память: `memory_max_chars 200–2000 + сохранение/persist (dev-mode)`

- Where: та же dev-панель → «максимум символов памяти на запрос».
- Steps: установи значение (шаг 50, диапазон 200–2000); нажми «Сохранить»; закрой и переоткрой панель.
- Expected: при сохранении уходит ОДИН combined-POST `/api/userconfig` с `modes + aux_models + protected_recent + memory_max_chars`; появляется «Сохранено»; после переоткрытия значения подтянуты из `/api/userconfig` (а не сброшены к дефолтам 6 / 600).

☐ pass

T-SET-23 — Открытие каталога моделей

- Where: Модели и режимы → «Каталог моделей».
- Steps: открой каталог.
- Expected: модалка; шапка «Каталог моделей» + счётчик «N моделей»; чипы-счётчики (зрение/код/рассуждение/роллплей/картинки); поиск; категории-лента; ряд тумблеров-фильтров + сортировки; ряд пресет/сохранённых чипов; сетка карточек.

☐ pass

T-SET-24 — Глубокий фильтр: Контекст $\geq 128k$

- Where: каталог → селект «Контекст».
- Steps: выбери « $\geq 128k$ » (есть также « $\geq 32k$ » и « $\geq 1M$ »).
- Expected: селект-чип подсвечивается циан (не дефолт); сетка показывает только модели с `ctx  $\geq 128000$` ; счётчик «активно фильтров» растёт.

☐ pass

T-SET-25 — Глубокий фильтр: Цена (бесплатно / дешево / премиум)

- Where: каталог → селект «Цена».
- Steps: пройди «бесплатно», затем «дешево» ($\leq 1\$/1k$), затем «премиум» ($>1\$/1k$).
- Expected: каждый раз набор карточек меняется соответственно; чип подсвечен.

☐ pass

T-SET-26 — Глубокий фильтр: Свобода ($\geq 50\%$ / $\geq 90\%$)

- Where: каталог → селект «Свобода».
- Steps: выбери « $\geq 50\%$ », затем « $\geq 90\%$ ».
- Expected: остаются модели с `uncensored_pct  $\geq$  порога` (булев флаг трактуется как $\sim 85\%$); у карточек « $\geq 90\%$ » виден бейдж «N% без цензуры».

☐ pass

T-SET-27 — Глубокий фильтр: Тип/ модальность (текст / картинки / голос)

- Where: каталог → селект «Тип».
- Steps: выбери «картинки», затем «голос», затем «текст».
- Expected: сетка сужается по модальности (видео в данных нет — только три типа).

☐ pass

T-SET-28 — Тумблеры-фильтры (без цензуры / TEE / дёшево / RU / бесплатные)

- Where: каталог → ряд тумблеров.
- Steps: включай по одному.
- Expected: каждый тумблер красится своим тоном (без цензуры=rose, TEE=emerald, дёшево=amber, RU=sky, бесплатные=emerald); фильтры комбинируются по И.

☐ pass

T-SET-29 — Сохранить фильтр-чип, переприменить, удалить

- Where: каталог → строка пресет/чипов.
- Steps: набери комбинацию фильтров; нажми «сохранить фильтр»; во всплывшем `prompt` введи имя (напр. «Мой фильтр»); затем нажми «сбросить всё»; кликни сохранённый чип; затем удали его крестиком на чипе.
- Expected: при 0 активных фильтров кнопка «сохранить фильтр» неактивна. После сохранения появляется фиолетовый чип с именем (`persist в taiga.catalog.savedFilters`); клик переприменяет ту же комбинацию; крестик удаляет чип; пережив перезагрузку (`localStorage`).

☐ pass

T-SET-30 — Встроенные пресет-чипы

- Where: каталог → пресеты «Дешёвые», «Длинный контекст», «Без цензуры».
- Steps: кликни каждый.
- Expected: применяется соответствующая комбинация (Дешёвые → цена «дёшево»; Длинный контекст → контекст $\geq 128k$; Без цензуры → тумблер «без цензуры» + свобода $\geq 90\%$). Пресеты не удаляются.

☐ pass

T-SET-31 — Клик по бренду карточки → фильтр + новинки вперёд

- Where: каталог → имя бренда-провайдера под названием модели в карточке.
- Steps: кликни бренд (напр. «openrouter»).
- Expected: появляется циан-чип бренда со «сбросить»; сетка — только модели этого бренда, новинки (по `created`) первыми; повторный клик по бренду сбрасывает фильтр.

☐ pass

T-SET-32 — Сортировки витрины

- Where: каталог → правый ряд «по уму / дешевле / новые / контекст».
- Steps: переключай сортировки.
- Expected: порядок карточек меняется (по уму=smart↓, дешевле=free→цена↑, новые=created↓, контекст=ctx↓); активная подсвечена.

☐ pass

T-SET-33 — Полоска здоровья провайдеров (up/down/латентность)

- Where: каталог, под поиском (полоска показывается только если `/api/providers` вернул статусы).
- Steps: осмотри полоску «провайдеры N/M в строю».
- Expected: каждый провайдер — точка (зелёная пульсирующая = up, красная = down), имя, латентность «мс», «N назад» (на $\geq sm$). Лежащие провайдеры показаны первыми и красным. Если ручки нет (404) — полоски просто нет, каталог работает.

☐ pass

T-SET-34 — Притушивание моделей лежащего провайдера + «скрыть лежащих»

- Where: каталог (при наличии down-провайдера).
- Steps: найди модель down-провайдера; затем нажми «скрыть лежащих (N)» в полоске.
- Expected: карточки down-провайдера притушены (opacity ~50%) с бейджем «провайдер недоступен»; кнопка-тумблер скрывает/возвращает их модели (`aria-pressed`).

☐ pass

T-SET-35 — Per-model здоровье (бан 502/down скрыт)

- Where: каталог, сетка.
- Steps: сравни число «N моделей» с числом моделей в пикере.
- Expected: модели в бане (502/down, через `healthyOnly`) не показываются в сетке вообще (это отдельно от уровня провайдера).

☐ pass

T-SET-36 — Модель на каждый процесс / режим разработчика (тумблер)

- Where: Модели и режимы.
- Steps: кликни «Модель на каждый процесс (чат / код / ультра / картинки...)»; затем переключи тумблер «Доступ к файлам и терминалу».
- Expected: первая — панель настроек закрывается, открывается панель моделей-на-процесс (`onOpenModels`). Тумблер dev-mode красится `rose`, когда включён (`onDevMode`). У не-владельца тумблер задизейблен с подписью «Доступно только владельцу аккаунта».

☐ pass

T-SET-37 — Dev-mode per-mode: модель / max-tokens / температура / aux-модели

- Where: dev-панель (см. T-SET-21 как открыть).
- Steps: выбери режим (чат/код/...); впиши `id` модели; подвигай слайдер «макс. длина ответа» (256–32768); подвигай «температура» (0–1.5); впиши `id` в одну из «вспомогательных моделей»; нажми «Сохранить»; затем «сбросить <<режим>>».
- Expected: у режимов с правками появляется точка «*»; значения подписаны `live` (ток. / число / авто); «сброс» температуры → «авто»; «сбросить режим» очищает все правки этого режима; «Сохранено» после сохранения.

☐ pass

Theme

T-SET-38 — Схема: Светлая / Тёмная / Системная — применяется live

- Where: Оформление → «Тема и акцент» → секция «Режим».
- Steps: переключи Светлая → Тёмная → Системная.
- Expected: интерфейс мгновенно перекрашивается (класс `.dark` / `data-scheme="light"` на `<html>`); «Системная» начинает следовать системной теме ОС вживую.

☐ pass

T-SET-39 — Акцент — применяется live и persists

- Where: Тема и акцент → секция «Акцент».
- Steps: выбери палитру (Полночь / Слейт / Киберпанк / Тайга); закрой, перезагрузи.
- Expected: акцент (`--taiga-accent`) меняется сразу в превью и по всему UI; выбор сохраняется (`taiga.theme`) и переживает перезагрузку.

☐ pass

T-SET-40 — Шрифт интерфейса

- Where: Тема и акцент → секция «Шрифт».
- Steps: выбери Гист / Анбаундед / Руссо / Моно.
- Expected: шрифт всего интерфейса меняется сразу (`--font-app`); галка на выбранном; persists (`taiga.font`).

☐ pass

Usage&Billing

T-SET-41 — Аналитика расходов: периоды + per-model

- Where: Использование и подписка → «Аналитика расходов».
- Steps: открой; переключи период «Сегодня / 7 дней / 30 дней»; переключи метрику «трата / запросы»; переключи вид «Бары / Таблица».
- Expected: три стат-карточки (Потрачено $\text{R/\$} \cdot \text{Запросов} \cdot \text{Токенов}$) пересчитываются; «По моделям» показывает бары или таблицу (Модель / Запросов / Токенов / R). Если за период нет журнала — «Пока нет данных об использовании».

☐ pass

T-SET-42 — Экспорт CSV скачивается

- Where: Аналитика расходов → кнопка «CSV».
- Steps: при наличии данных нажми «CSV».
- Expected: скачивается файл `taiga-usage-<период>-<дата>.csv` с заголовком (datetime, model, kind, in_tokens, out_tokens, total_tokens, cost_usd, charge_usd) и BOM для кириллицы. При пустом периоде кнопка задизейблена.

☐ pass

T-SET-43 — Подписка — лимиты (sub-meters) (owner-only)

- Where: Использование и подписка → «Подписка — лимиты».
- Steps: открой; осмотри бары и таймеры.
- Expected: бар «Картинки сегодня» (N/лимит, % , осталось), «Токены за неделю» (или заметка, что бэк ещё не отдаёт недельный счётчик, лимит 60M), два живых обратных отсчёта «До сброса картинок» (до полуночи) и «До сброса токенов» (до period_end/понедельника), тикают по секундам; кнопка «Обновить» (свежие счётчики из `/api/balances?refresh=1`). Если подписка не активна — «подписка не активна».

☐ pass · (owner-only · серверный гейт: у не-владельца панель = `null`)

Diagnostics

T-SET-44 — Диагностика: пинг бэкенда (owner-only)

- Where: Использование и подписка → «Диагностика и здоровье».
- Steps: открой панель.
- Expected: секция «бэкенд» — строка `/api/init` «отвечает / N мс» (зелёная при ok); строка «Модели — N в витрине · M всего»; кнопка «обновить» перепроверяет. Если бэк не отвечает — состояние «Бэкенд не отвечает» с кнопкой «повторить».

☐ pass · (owner-only · серверный гейт)

T-SET-45 — Диагностика: 5 кошельков + total (owner-only)

- Where: Диагностика → секция «кошельки».
- Steps: осмотри список кошельков.
- Expected: до 5 кошельков в порядке Venice, NanoGPT, Chutes, Redpill (TEE), AIMLAPI (медиа); у каждого точка (зелёная/янтарь при низком/серая при «нет ключа») и баланс ₺ либо «баланс в дашборде»/«—»; внизу «Всего на счетах» с totalом (`_total_usd`).

☐ pass · (owner-only)

T-SET-46 — Диагностика: самопроверка → 6 чеков (owner-only)

- Where: Диагностика → секция «самопроверка».
- Steps: нажми «Запустить самопроверку».
- Expected: кнопка → «Гоняю проверку...»; затем приходит `/api/selftest` со списком чеков (каталог, биллинг, RAG, поиск, планировщик, вызов модели — ~6 шт), у каждого зелёная/красная галка + «N мс» + опц. detail; вверху бейдж «passed/total · total_ms»; кнопка превращается в «Прогнать ещё раз». 403 у не-владельца показывается мягко («доступна только владельцу»).

☐ pass · (owner-only)

Integrations

T-SET-47 — MCP: открытие маркетплейса и список

- Where: Интеграции (MCP / BYOK) → «Внешние инструменты (MCP)».
- Steps: открой панель (выезжает справа).
- Expected: секция «маркетплейс — подключи одной кнопкой» со списком коннекторов (бейдж категории + бейдж «готов / нужен URL / нужен аккаунт»); секция «подключено» (если пусто — «Пока ничего...»); сворачиваемая «свой сервер по URL».

☐ pass

T-SET-48 — MCP: подключить коннектор по URL (UI-флоу)

- Where: MCP → коннектор с бейджем «нужен URL» (auth=url).
- Steps: впиши URL в поле под коннектором; нажми «подключить».
- Expected: пока URL пуст — «подключить» задизейблена; после ввода активна; по клику крутится спиннер, затем сверху баннер-сообщение (зелёный ок «N инструментов» или янтарный с ошибкой). НЕ нужно реально иметь живой MCP — проверяется валидация/состояние кнопки и сообщение.

☐ pass

T-SET-49 — MCP: коннектор с OAuth-токеном (UI-флоу, без реального токена)

- Where: MCP → коннектор «нужен аккаунт» (auth=oauth, напр. GitHub/Notion).
- Steps: проверь, что появляется поле «Personal access token» (type=password) с замочком и подписью «хранится зашифровано».
- Expected: «подключить» задизейблена, пока токен пуст; поле скрывает символы; не вставляй реальный секрет — достаточно убедиться, что флоу/валидация работают (пустой/непустой).

☐ pass

T-SET-50 — MCP: свой сервер по URL + удаление

- Where: MCP → «свой сервер по URL».
- Steps: разверни секцию; впиши «имя» и «URL»; нажми «подключить»; затем в списке «подключено» нажми удалить (корзина) и тумблер питания (Power).
- Expected: «подключить» активна только при заполненных имя+URL; добавленный сервер появляется в «подключено» (точка статуса, число инструментов/ресурсов/промптов); Power включает/выключает; корзина удаляет.

☐ pass

T-SET-51 — Мои API-ключи (mostik-sk): создать и отозвать

- Where: Интеграции → «мои API-ключи (mostik-sk)».
- Steps: нажми «создать ключ»; скопируй свежий ключ кнопкой-копи; затем отзови ключ корзиной (появляется при ховере на строке).
- Expected: показывается база `http://127.0.0.1:8777/v1`; свежий ключ выводится в зелёном блоке с копированием (галка после копи); список существующих ключей; корзина отзывает (POST `/api/apikeys` action=revoke).

☐ pass

T-SET-52 — ВЮК: ввод ключа провайдера (UI-флоу, без реального ключа)

- Where: Интеграции → «мои ключи провайдеров (ВЮК)».
- Steps: выбери провайдера в селекте (Venice / NanoGPT / Chutes / Redpill (TEE)); вставь в поле тестовую строку; нажми «Сохранить»; затем удали сохранённый ключ корзиной.
- Expected: «Сохранить» задизейблена при пустом поле; после сохранения провайдер появляется в списке «— ключ сохранён» (зелёным), сообщение «Ключ сохранён»; корзина → «Ключ удалён». Поле type=password, autocomplete off. Реальный секрет не нужен — проверяется флоу/валидация.

☐ pass

Misc

T-SET-53 — Бэкап: экспорт конфига в файл

- Where: Разное · Что нового → «бэкап и перенос».
- Steps: нажми «Экспорт в файл».
- Expected: скачивается `taiga-backup.json` (только ключи `taiga.*`, БЕЗ переписки — `taiga.chat` / `taiga.tree` / `taiga.chats` исключены); сообщение «Экспортировано».

☐ pass

T-SET-54 — Импорт конфига из файла

- Where: «бэкап и перенос» → «Импорт из файла».
- Steps: выбери ранее экспортированный `taiga-backup.json`.
- Expected: сообщение «Импортировано N — перезагрузка...», затем авто-reload. Битый/чужой файл → «Файл не распознан» / «Битый файл бэкапа» без reload.

☐ pass

T-SET-55 — Что нового: версионированный чейнджлог

- Where: Разное · Что нового → «Что нового».
- Steps: открой панель.
- Expected: лента релизов сверху-новее (2.6.0 → 2.0.0); каждый релиз — заголовок, `vX.Y.Z`, дата (рус. формат), список пунктов с тегами «новое / улучшено / исправлено»; первые 2 релиза раскрыты, остальные под «Прошлые релизы (N)».

☐ pass

T-SET-56 — «Новое» бейджи с прошлого захода

- Where: панель «Что нового».
- Steps: закрой панель (фиксирует последнюю версию в `taiga.whatsNew.seen`); открой снова.
- Expected: при первом заходе релизы новее виденной версии помечены бейджем «новое» и счётчиком в шапке «N новых»; после закрытия (всё прочитано) при повторном открытии бейджи и счётчик пропадают; точка «новое» есть и на свёрнутом «Прошлые релизы», если там есть непрочитанные.

☐ pass

T-SET-57 — Воркфлоу / Фоновые задачи открываются

- Where: Студия и медиа → «Воркфлоу» и «Фоновые задачи».
- Steps: открой обе панели.
- Expected: каждая открывается; «Фоновые задачи» показывает секции «Выполняется» и «Завершено» со счётчиками; при отсутствии задач — «Пока нет фоновых задач».

☐ pass

T-SET-58 — Фоновые задачи: running / finished

- Where: «Фоновые задачи».
- Steps: запусти фоновую работу (напр. прогон оркестратора или генерацию медиа), вернись в панель.
- Expected: активная задача в «Выполняется» (бейдж «выполняется» со спиннером, тикающее «прошло»); по завершении уходит в «Завершено» с «готово»/«ошибка». Расписания (crop) показаны как «по расписанию»/«на паузе» с «следующий через N».

☐ pass

T-SET-59 — Голос: озвучка (TTS) и язык диктовки

- Where: Студия и медиа → «голос — озвучка и диктовка».
- Steps: переключи тумблер «Озвучивать ответы вслух»; выбери провайдера TTS (Бесплатно (Google) / Платно); смени «Язык голосового ввода» в селекте.
- Expected: все три сохраняются в voice-pref; тумблер красится циан; выбор языка из списка STT_LANGS .

☐ pass

T-SET-60 — Тосты: успех и ошибка

- Where: любое действие, кидающее тост (напр. в Image Studio «soon», или ошибка сети).
- Steps: спровоцируй success-тост и error-тост.
- Expected: тост всплывает снизу-справа; success = зелёная галка, error = rose-треугольник, info = акцент; авто-закрытие по таймеру; крестик закрывает вручную; не больше 4 одновременно.

☐ pass

T-SET-61 — Онбординг: пустое состояние

- Where: новый аккаунт без чатов (или очисти `taiga.onboarded` в localStorage и перезагрузи).
- Steps: открой `/app` без активного чата.
- Expected: блок «Что умеет Тайга» с 9 карточками-возможностями (Совет моделей, Сравнение, Мозг и Ресёрч, Картинки, Видео-Музыка-3D, Навыки и агенты, Память, MCP-коннекторы, Файлы и зрение) + чипы «Попробуй спросить — по режимам». Тап по карточке/чипу сеет пример-промт в композер и помечает онбординг пройденным.

☐ pass

T-SET-62 — «Пройти тур» (capability-tour)

- Where: онбординг → кнопка «Пройти тур».
- Steps: нажми «Пройти тур»; листай «Дальше»/«Назад»; на шаге с промптом нажми кнопку-действие (напр. «Собрать совет»); проверь Esc.
- Expected: модалка из 5 шагов с прогресс-полоской «Тур · i из 5»; «Дальше/Назад» (и стрелки ←/→); «попробовать» сеет промпт в композер и закрывает тур; Esc/клик по фону/«Пропустить» закрывают; прохождение запоминается (`taiga.tourDone`), сам больше не всплывает.

☐ pass

Accessibility

T-SET-63 — Клавиатурный обход настроек целиком

- Where: открытая панель настроек.
- Steps: только клавиатурой (Tab/Shift+Tab/стрелки/Enter/Space) пройди: поиск → группы → блоки контента → кнопки/тумблеры/селекты.
- Expected: все интерактивные элементы достижимы и активируются с клавиатуры; нет ловушек фокуса; глубокие фильтры каталога — нативные `<select>` (доступны из коробки).

☐ pass

T-SET-64 — Видимые фокус-кольца

- Where: панель настроек и каталог.
- Steps: таб по элементам, следи за фокусом.
- Expected: у крестика закрытия, очистки поиска, пунктов навигации и т.п. видны focus-кольца (`focus-visible:ring`). Карточки онбординга/каталога имеют акцентное фокус-кольцо.

☐ pass

T-SET-65 — Screen-reader: live-регион поиска и тостов

- Where: панель настроек / тостер.
- Steps: с включённым screen-reader (VoiceOver) выполни поиск настройки и спровоцируй тост. Спот-чек без SR: в DevTools проверь наличие `aria-live="polite"` на счётчике результатов поиска и `role="status"` `aria-live="polite"` на контейнере тостов.
- Expected: счётчик «Найдено: N / Ничего не найдено» озвучивается; тосты озвучиваются (контейнер `role="status"`); кнопки имеют `aria-label` (Заккрыть настройки, Очистить поиск, Заккрыть уведомление и т.п.).

☐ pass

T-SET-66 — Reduced-motion уважается

- Where: ОС → включи «Reduce motion».
- Steps: открой «Что нового», каталог (пульс точек провайдеров), тур, тосты.
- Expected: декоративные анимации глушатся (пульс точки `motion-reduce:hidden`, спиннеры `motion-reduce:animate-none`, переходы кросс-фейдом); смысловые счётчики (sub-meters таймеры) продолжают тикать.

☐ pass

Mobile

T-SET-67 — Навигация настроек сворачивается в ленту-табы

- Where: узкий выюпорт ($\leq 768px$), панель настроек.
- Steps: открой настройки на мобильном размере.
- Expected: панель на весь экран (`h-100dvh`); левое меню групп превращается в горизонтальную скролл-ленту табов сверху; контент ниже. При активном поиске лента скрыта.

☐ pass

T-SET-68 — Composer и safe-area на мобиле

- Where: узкий выюпорт, главный экран чата + панели.
- Steps: осмотри низ панелей и композер на устройстве с «чёлкой»/жестовой панелью.
- Expected: нижний паддинг учитывает `env(safe-area-inset-bottom)` (контент не залезает под системную панель — заложено в `settings/whats-new/custom-instructions/mcp/onboarding`); композер доступен.

☐ pass

T-SET-69 — Нет горизонтального переполнения, панели юзабельны

- Where: узкий выюпорт.
- Steps: пройди каталог моделей, диагностику, аналитику, sub-meters, tasks на мобиле.
- Expected: нет горизонтального скролла страницы; сетка каталога в 1 колонку; ленты чипов/категорий скроллятся внутри (скрытый скроллбар); модалки на весь экран (`100dvh`), кнопки тапабельны (мин. размеры 9–10).

☐ pass

T-SET-70 — MCP-панель как выезжающий drawer

- Where: узкий выюпорт → MCP.
- Steps: открой «Внешние инструменты (MCP)».
- Expected: на мобиле — на весь экран; на десктопе — выезжает справа (`md:max-w-xl`, `md:border-l`); закрытие крестиком/фоном.

☐ pass

4 · Бэкенд, данные и безопасность

Тайга ИИ — QA Test

Plan: Backend / Data /

Ops / Security

Manual, founder-run (curl + observing). Backend: stdlib Python `server.py` on `:8777` (`PORT = 8777`, `server.py:51`). `default = OWNER ( is_owner`, `server.py:5213` — a uid is owner iff its user record has `owner:true` or the uid is literally `"default"`). A different uid (e.g. `stranger123`) is a non-owner.

Conventions

- `BASE=http://127.0.0.1:8777` — set once: `export BASE=http://127.0.0.1:8777`
- OWNER tests pass `"user":"default"`; non-owner tests pass `"user":"stranger123"`.
- Start the backend first: `python3 server.py` (from repo root) → logs to stderr; in practice it is run as `python3 server.py > server.out 2>&1` so the `req ...` log lines and tracebacks land in `server.out`.
- "Provider dry" (no key / no balance / overload) is **not a failure** — those tests note a skip condition exactly like the automated suite does.

All endpoints, params, gates, and limits below were verified against `server.py` / `scheduler.py` and are cited `file:line`.

Automated (gating suites)

These must pass before manual testing is meaningful. Run from repo root.

T-BE-01 — Python endpoint smoke suite (16 tests)

- Command/where: backend running, then

```
cd /Users/damir12/Downloads/claude-
sessions/2026-06-10/mostik-ai
python3 -m unittest tests.test_endpoints -
v
```

- Expected: OK — 16 tests run
(`tests/test_endpoints.py`: `TestInit` 1, `TestChat` 2, `TestCouncil` 1, `TestSearchChats` 1, `TestWorkflow` 2, `TestSelftest` 1, `TestRag` 1, `TestOrchestrate` 2, `TestOwnerGates` 3, `TestInputValidation` 2 = 16). Some may report `skipped` when a provider is dry (chat/rag/orchestrate/workflow) — that is green, not a failure. **Zero failures/errors**. Full run ~1–2 min (`tests/README.md`).

☐ pass

T-BE-02 — Frontend unit suite (73 tests)

- Command/where:

```
cd /Users/damir12/Downloads/claude-
sessions/2026-06-10/mostik-
ai/taiga-web
npx vitest run
```

- Expected: all green across 5 files (`usage-log` , `money` , `msg-tree` , `rag` , `decision`), **73 tests passed**, 0 failed.

☐ pass

T-BE-03 — TypeScript typecheck

- Command/where: `cd taiga-web && npx tsc --noEmit`
- Expected: exits 0, no type errors printed.

☐ pass

T-BE-04 — Production build

- Command/where: `cd taiga-web && npm run build` (= `next build` , per `package.json`)
- Expected: build completes successfully, no errors.

☐ pass

Endpoints (smokes)

T-BE-05 — /api/init returns 200 + all keys

- Command/where:

```
curl -s "$BASE/api/init?user=default" |
python3 -m json.tool | head -40
```

- Expected: HTTP 200, JSON containing `users` , `models` , `full` , `system` , `relay_craft` , `keys` , `balance` , `balances` , `billing` , `byok` , `apikeyes` , `api_base` , `settings` , `memory` (server.py:8928). `keys` is `{venice,nanogpt,chutes,redpill}` → bool key-present. `billing.owner == true` for `default` . `api_base = http://127.0.0.1:8777/v1` .

☐ pass

T-BE-06 — Chat SSE event order meta → delta → cost → done + served_by

- Command/where:

```
curl -sN -X POST "$BASE/api/chat" -H
'Content-Type: application/json' \
-d '{"user":"default","model":"__auto__","messages":
[{"role":"user","content":"Скажи ровно:
привет"}]}'
```

- Expected: text/event-stream . Events in order:
{"type":"meta","model":...} (server.py:10293) →
one+ {"type":"delta","text":...} →
{"type":"cost","owner":true,...}
(server.py:10618) → {"type":"done","served_by":
{"provider":..., "model":...}} (server.py:10628). For
OWNER, cost.owner=true and balance is **not**
deducted. If provider dry → a single
{"type":"error",...} → skip.

☐ pass

T-BE-07 — search_chats POST shape

- Command/where:

```
curl -s -X POST "$BASE/api/search_chats" -
H 'Content-Type: application/json'
\
-d '{"user":"default","q":"rect"}'
```

- Expected: 200, {"results":
[...], "count":N, "q":"rect"} with count ==
len(results) , q echoed (server.py:9072). Owner-
only all:true widens to all users' chats (owner =
is_owner(uid) and bool(c.get("all")) ,
server.py:9077).

☐ pass

T-BE-08 — workflow GET lists templates

- Command/where: curl -s "\$BASE/api/workflow"
| python3 -m json.tool
- Expected: 200, {"templates":[...]} (server.py:9026).
4 built-ins present (server.py:1810): research-brief ,
image-from-idea , doc-qa , rewrite-polish ;
each has id,title,desc,steps .

☐ pass

T-BE-09 — workflow POST runs a single-step template

- Command/where:

```
curl -s -X POST "$BASE/api/workflow" -H
  'Content-Type: application/json' \
  -d '{"user":"default","template_id":"rewrite-
    polish","input":"перепеши это: перевод как
    дила"}'
```

- Expected: HTTP 200 always (errors are in-body, server.py:8770). On success `{"ok":true,"steps":[...],"result":...}`, last step has `output`. Provider dry → `ok:false` with a balance/no-key message → skip.

☐ pass

T-BE-10 — orchestrate returns final + plan + results + steps

- Command/where:

```
curl -s -X POST "$BASE/api/orchestrate" -H
  'Content-Type: application/json' \
  -d '{"user":"default","task":"В одном
    предложении: что такое HTTP?"}'
```

- Expected: 200 with `{plan, results, final, steps}`, non-empty `final` (server.py:8690, handler `api_orchestrate`). Owner not charged. Non-owner is balance-gated (needs $\sim \$0.05 \times \text{markup}$, else 402, server.py:8705). HTTP 402/502/503 or empty-provider error → skip.

☐ pass

T-BE-11 — orchestrate streaming timeline (optional)

- Command/where:

```
curl -sN -X POST "$BASE/api/orchestrate" -
  H 'Content-Type: application/json'
  \
  -d '{"user":"default","task":"кратко:
    что такое DNS?","stream":true}'
```

- Expected: `text/event-stream` of `{"kind":...}` events ending with `{"kind":"done","final":...,"plan":...,"results":...}` (server.py:8720-8742).

☐ pass

T-BE-12 — RAG ingest → query → delete round-trip

- Command/where:

```
# ingest
curl -s -X POST "$BASE/api/rag_ingest" -H
'Content-Type: application/json' \
-d '{"user":"default","name":"taiga_smoke_doc.txt","text":"Тестовый
документ. Секретное кодовое слово: оранжевый носорог. Наполнение
для эмбедингов."}'

# query
curl -s -X POST "$BASE/api/rag_query" -H
'Content-Type: application/json' \
-d '{"user":"default","query":"кодовое
слово оранжевый носорог","k":4}'

# delete (cleanup)
curl -s -X POST "$BASE/api/rag_delete" -H
'Content-Type: application/json' \
-d '{"user":"default","name":"taiga_smoke_doc.txt"}
```

- Expected: ingest →
{ "ok":true,"doc":...,"chunks":>=1,"source":null
,"workspace":...,"docs":[...] } (server.py:8345).
query → { "hits":[...],"docs":[...] } with ≥1 hit
whose top has doc + text (server.py:8374). delete
→ { "ok":true,"docs":[...] } (server.py:8401).
Embedding-provider down → 502 → skip.

☐ pass

T-BE-13 — providers health snapshot (4 providers)

- Command/where: `curl -s "$BASE/api/providers"`
| `python3 -m json.tool`
- Expected: 200, { "providers":
[...], "threshold":3, "cooldown_sec":120.0, "now":...
} (server.py:8981, HEALTH_FAIL_THRESHOLD=3
server.py:116, HEALTH_COOLDOWN_SEC=120.0
server.py:117). `providers` lists exactly the 4
commission providers — `venice`, `nanogpt`,
`chutes`, `redpill` (`PROVIDERS`, server.py:54-75) —
each with {name, ok, configured, last_checked,
latency_ms, consecutive_fails, degraded,
last_error} (server.py:178). Healthy → `ok:true`,
`degraded:false`, `consecutive_fails:0`.
`configured` reflects key-file presence.

☐ pass

T-BE-14 — selftest 6/6 (owner)

- Command/where:

```
curl -s -X POST "$BASE/api/selftest" -H
'Content-Type: application/json' -d
'{"user":"default"}'
```

- Expected: 200, {ok, passed, failed, total_ms, checks[] } (server.py:8665). **6 checks**
(server.py:8632-8662): "Каталог моделей", "Биллинг и баланс", "База знаний (RAG)", "Супер-поиск", "Планировщик агентов", "Вызов модели". Each check has {name,ok,ms,detail} ; passed+failed == 6 . Healthy box → ok:true, passed:6, failed:0 (check #6 reports a clean "пропуск" if no provider key — still counts as ok).

☐ pass

T-BE-15 — /v1/models OpenAI-compatible catalog list

- Command/where: `curl -s "$BASE/v1/models" | python3 -m json.tool | head`
- Expected: 200, {"object":"list","data":[{"id":...,"object":"model","owned_by":...}, ...]} (server.py:8991), one row per RICH model.

☐ pass

Owner-gating (run as stranger123)

T-BE-16 — selftest non-owner → 403

- Command/where: `curl -s -o /dev/null -w "%{http_code}\n" -X POST "$BASE/api/selftest" -H 'Content-Type: application/json' -d '{"user":"stranger123"}'`
- Expected: 403 , body {"error":"Самопроверка /sprint — только владелец.", "code":"forbidden"} (server.py:8593-8594; code added by envelope server.py:7705).

☐ pass (owner-only)

T-BE-17 — memory_consolidate non-owner → 403

- Command/where: `curl -s -X POST "$BASE/api/memory_consolidate" -H 'Content-Type: application/json' -d '{"user":"stranger123"}'`
- Expected: 403, {"error":"Уплотнение памяти — только владелец.", "code":"forbidden"} (server.py:8682-8683).

☐ pass (owner-only)

T-BE-18 — /api/run (code interpreter) non-owner → 403 (anti-RCE)

- Command/where: `curl -s -X POST "$BASE/api/run" -H 'Content-Type: application/json' -d '{"user":"stranger123","code":"print(1)","lang":"python"}'`
- Expected: 403, {"error":"Запуск кода — только владелец.", "code":"forbidden"} (server.py:9198-9199).

☐ pass (owner-only)

T-BE-19 — /api/billing write non-owner → 403

- Command/where: `curl -s -X POST "$BASE/api/billing" -H 'Content-Type: application/json' -d '{"user":"stranger123","action":"set_markup","markup_pct":0}'`
- Expected: 403, {"error":"только владелец", "code":"forbidden"} (server.py:9362-9363). Confirms no non-owner can change markup / rates / enabled / top-up others.

☐ pass (owner-only)

T-BE-20 — /api/identity POST non-owner → 403

- Command/where: `curl -s -X POST "$BASE/api/identity" -H 'Content-Type: application/json' -d '{"user":"stranger123","persona":"ты пират"}'`
- Expected: 403, {"error":"Менять личность Тайги может только владелец.", "code":"forbidden"} (server.py:9204-9205). (GET /api/identity is open and returns persona/default/name, server.py:8962.)

☐ pass (owner-only)

T-BE-21 — /api/catalog_refresh POST non-owner → 403

- Command/where: `curl -s -X POST "$BASE/api/catalog_refresh" -H 'Content-Type: application/json' -d '{"user":"stranger123"}'`
- Expected: 403, `{"error":"только владелец","code":"forbidden"}` (server.py:8546-8547).

☐ pass (owner-only)

T-BE-22 — Non-owner CAN do a normal chat

- Command/where:

```
curl -sN -X POST "$BASE/api/chat" -H
'Content-Type: application/json' \
-d '{"user":"stranger123","model":"__auto__","messages":
[{"role":"user","content":"привет"}]}'
```
- Expected: NOT 403. Either a normal SSE `meta → delta → cost → done` stream (with `cost.owner=false` and a `balance` field, server.py:9664/10615), OR — if the stranger has \$0 balance and billing is on — an SSE `{"type":"error","message":"Баланс исчерпан. Пополни счёт..."}` (server.py:10286-10288). Both confirm chat is open to non-owners (gate is balance, not ownership).

☐ pass

Security

T-BE-23 — Per-IP rate limit → 429 + Retry-After (non-owner)

- Command/where: fire >30 expensive (POST) requests from one IP inside the 10s burst window as a NON-owner (`RL_IP_BURST=30` , `RL_IP_BURST_WINDOW=10` , `server.py:2069-2070`):

```
for i in $(seq 1 35); do
  curl -s -o /dev/null -w "%{http_code} " \
    -X POST "$BASE/api/orchestrate" \
    -H 'Content-Type: application/json' -d \
    '{"user":"stranger123","task":"x"}'
done; echo
# show the headers of one rejected call:
curl -s -D - -o /dev/null -X POST \
  "$BASE/api/orchestrate" \
  -H 'Content-Type: application/json' -d \
  '{"user":"stranger123","task":"x"}'
```

- Expected: after ~30 hits the codes flip to `429` . The `429` response carries a `Retry-After:` header (seconds) and body `{"error": "...", "retry_after": N}` (`_ip_guard` , `server.py:7734-7746`; `rate_ip_ok` , `server.py:2080`). Note: a sustained cap also exists (`RL_IP_SUSTAINED=120` /60s, `server.py:2071-2072`).

- ☐ pass (destructive — careful; trips the limiter for that IP for a few seconds)

T-BE-24 — Owner is exempt from per-IP limit

- Command/where: same loop as T-BE-23 but `"user":"default"` against `/api/orchestrate` or `/api/chat` .
- Expected: **never** `429` from the IP limiter — `RL_IP_OWNER_EXEMPT=True` and `_ip_guard` / `_ip_guard_sse` short-circuit for owners (`server.py:2073`, `7738`, `7756`). (Responses may still be `200/402/502` by provider state, but not `429-rate_limited`.)

- ☐ pass (owner-only)

T-BE-25 — Input validation: too many messages → 400

- Command/where:

```
python3 - <<'PY'
import json,urllib.request
msgs=[{"role":"user","content":"x"} for _
      in range(401)]
d=json.dumps({"user":"default","messages":msgs}).encode()
r=urllib.request.Request("http://127.0.0.1:8777/api/chat",data=d,headers=
{"Content-Type":"application/json"})
try: urllib.request.urlopen(r)
except urllib.error.HTTPError as e:
    print(e.code, e.read().decode())
PY
```

- Expected: 400, error mentions "слишком много сообщений (>400)" (SEC_MAX_MESSAGES=400 , server.py:2111; _sec_messages_ok server.py:2121; chat gate server.py:10104). Rejected before any provider/billing call.

☐ pass

T-BE-26 — Input validation: >4MB total text → 400

- Command/where:

```
python3 - <<'PY'
import json,urllib.request,urllib.error
big="a"*4_100_000 # >
SEC_MAX_TOTAL_CHARS = 4_000_000
d=json.dumps({"user":"default","messages":
[{"role":"user","content":big}]}).encode()
r=urllib.request.Request("http://127.0.0.1:8777/api/chat",data=d,headers=
{"Content-Type":"application/json"})
try: urllib.request.urlopen(r)
except urllib.error.HTTPError as e:
    print(e.code, e.read().decode()
[:120])
PY
```

- Expected: 400, error "слишком большой объём текста в сообщениях" (SEC_MAX_TOTAL_CHARS=4_000_000 , server.py:2112/2142).

☐ pass

T-BE-27 — Oversized RAG ingest → 400

- Command/where: post a doc whose `text` exceeds `SEC_MAX_RAG_TEXT_CHARS=8_000_000` (server.py:2114; gate server.py:8298-8299):

```
python3 - <<'PY'
import json,urllib.request,urllib.error
d=json.dumps({"user":"default","name":"huge.txt","text":"a"*8_100_000}).encode()
r=urllib.request.Request("http://127.0.0.1:8777/api/rag_ingest",data=d,headers=
    {"Content-Type":"application/json"})
try: urllib.request.urlopen(r)
except urllib.error.HTTPError as e:
    print(e.code, e.read().decode()
          [:120])
PY
```

- Expected: 400, `{"error":"документ слишком большой", "code":"bad_request"}` . (Binary ingest via `raw_b64` is separately capped at `SEC_MAX_RAG_RAW_BYTES=25_000_000` , server.py:2115/8306-8314 → "файл слишком большой".)

☐ pass

T-BE-28 — Abuse guard rejects universally-banned content

- Command/where: send a chat whose last user message trips `abuse_check` (the minors+sexual co-occurrence guard, server.py:2148-2165). Use placeholder banned terms only to confirm the 400 path, not real content.
- Expected: chat path returns an error (SSE `error` or 400) and the request is logged via `log_abuse` ; no provider call is made. (This protects the shared key from a provider ban — server.py comment 2147.)

☐ pass (careful — use the minimal trigger only)

Observability

T-BE-29 — Per-request log line format

- Command/where: with the server writing stderr to `server.out`, make any request then:

```
curl -s "$BASE/api/init?user=default"
      >/dev/null
tail -n 5 server.out
```

- Expected: a one-line `key=value` entry per request:
`req method=GET path=/api/init status=200 ms=`
`<n> uid=default (TAIGA_LOG default ON,`
`log_request server.py:2227-2245; emitted in`
`_dispatch finally, server.py:7731-7733). Secrets are`
never logged — only method/path/status/ms/uid (+ err
on failure).

☐ pass

T-BE-30 — Bad-JSON body → clean 400 {error,code:bad_json}

- Command/where:

```
curl -s -X POST "$BASE/api/chat" -H
      'Content-Type: application/json' -
      -data-binary '{not json}'
```

- Expected: 400, body `{"error":"тело должно быть корректным JSON","code":"bad_json"}`
(`_dispatch JSONDecodeError` branch,
`server.py:7663-7669`). No stack trace to client. Log line
shows `status=400 ... err_type=JSONDecodeError`.

☐ pass

T-BE-31 — Induced internal error → clean 500 {error,code:internal}, no stack trace

- Command/where: trigger an unhandled exception before any response is sent. A reliable inducer is a `/api/topup` with a non-numeric `rub` (forces a `float()` `ValueError` that is not pre-validated):

```
curl -s -X POST "$BASE/api/topup" -H
'Content-Type: application/json' -d '{"user":"default","rub":"abc"}'
```

- Expected: HTTP 500, body exactly `{"error": "внутренняя ошибка, попробуй ещё раз", "code": "internal"}` — NOT a Python traceback (`_dispatch` backstop, `server.py:7677-7687`). The real traceback goes to `server.out` (`stderr`) and the log line shows `status=500 ... err_type=ValueError`. (If a future patch pre-validates `rub`, use any other input that reaches an uncaught exception before headers are sent — the contract is: client sees clean 500 {error,code:internal}, `stderr` gets the trace.)

☐ pass

T-BE-32 — Client disconnect is swallowed (no crash)

- Command/where: start an SSE chat and kill it mid-stream:

```
curl -sN -X POST "$BASE/api/chat" -H
'Content-Type: application/json' \
-d '{"user":"default","model":"__auto__","messages":
  [{"role":"user","content":"напиши длинный
  текст"}]}' &
sleep 1; kill %1
```

- Expected: server stays up; `server.out` shows the request log line with `err_type=BrokenPipeError` (or `ConnectionResetError`) and `err="client disconnected"` — handled quietly, no second response attempted (`server.py:7670-7672`).

☐ pass

Providers (health tracker)

T-BE-33 — All 4 commission providers report ok + latency

- Command/where: `curl -s "$BASE/api/providers" | python3 -m json.tool`
- Expected: `providers[]` has the 4 entries `venice/nanogpt/chutes/redpill`. After normal use each shows `ok:true`, `degraded:false`, `consecutive_fails:0`, and a numeric `latency_ms` once a call has occurred (server.py:178-200). Providers without a key still appear with `configured:false`.

☐ pass

T-BE-34 — Degraded-provider semantics (document only — do NOT force an outage)

- Command/where: read-only; understand the contract from `/api/providers`.
- Expected: a provider flips to `ok:false`, `degraded:true` only after `HEALTH_FAIL_THRESHOLD=3` consecutive failures (server.py:116, snapshot logic server.py:188-189) and auto-recovers after `HEALTH_COOLDOWN_SEC=120s` (server.py:117). `last_error` carries the truncated last failure reason. One transient failure does NOT degrade a provider. **Do not deliberately break a key to test this.**

☐ pass (observe-only)

Caching / correctness

T-BE-35 — /api/init twice is byte-identical (catalog cache)

- Command/where:

```
curl -s "$BASE/api/init?user=default" > /tmp/init1.json
curl -s "$BASE/api/init?user=default" > /tmp/init2.json
diff /tmp/init1.json /tmp/init2.json && echo IDENTICAL
```

- Expected: `IDENTICAL` — the curated/full catalog payloads are served from a cache that is byte-stable between calls (`_catalog_payload_cached`, server.py:381; `curated_payload` / `full_catalog_payload`, server.py:1558/1571). (Balances embedded here are read live, not from that cache — see next test.)

☐ pass

T-BE-36 — Balance reflects a spend immediately (never cached stale)

- Command/where (owner test_topup path — needs test_topup=true set by owner first via /api/billing):

```
# owner enables test top-up
curl -s -X POST "$BASE/api/billing" -H
  'Content-Type: application/json' \
  -d '{"user":"default","action":"set_rates" "test_topup":true}'
  >/dev/null
# create a non-owner with a balance, read
balance
curl -s -X POST "$BASE/api/topup" -H
  'Content-Type: application/json' \
  -d '{"user":"stranger123","rub":1000}'
curl -s "$BASE/api/init?user=stranger123" | python3
  -c 'import
      sys,json;print("before",json.load(sys.stdin)
      ["billing"]["balance"])'
# do a paid chat as stranger (consumes
balance), then re-read
curl -sN -X POST "$BASE/api/chat" -H
  'Content-Type: application/json' \
  -d '{"user":"stranger123","model":"__auto__","messages":
      [{"role":"user","content":"привет"}]}' >/dev/null
curl -s "$BASE/api/init?user=stranger123" | python3
  -c 'import
      sys,json;print("after",json.load(sys.stdin)
      ["billing"]["balance"])'
```

- Expected: after balance < before balance immediately after the paid chat — balance is read live via user_balance(uid) on each init (server.py:8942-8944), and the chat's cost event carries the updated balance for non-owners (server.py:10615). Provider-level wallet balances refresh on demand via /api/balances?refresh=1 (server.py:8988, get_balances refresh path server.py:2671-2678). **Reset:** set_rates {"test_topup":false} when done.

- ☐ pass (owner-only setup; destructive — toggles test_topup, leave it OFF after)

Memory

T-BE-37 — Server memory grows after a memorable fact

- Command/where: write a fact via `/api/remember`, then re-read init memory:

```
curl -s "$BASE/api/init?user=default" | python3 -c
'import
sys,json;print("before",len(json.load(sys.stdin)
["memory"]))'
curl -s -X POST "$BASE/api/remember" -H
'Content-Type: application/json' \
-d '{"user":"default","messages":
[{"role":"user","content":"Запомни:
мой любимый цвет — оранжевый, и я
живу в Тбилиси."}]}'
curl -s "$BASE/api/init?user=default" | python3 -c
'import
sys,json;print("after",len(json.load(sys.stdin)
["memory"]))'
```

- Expected: `/api/remember` returns `{"memory": [...]}` with extracted facts (server.py:9090, `extract_memory`); init memory count after `>=` before (server.py:8949, `load_memory`). If extraction needs a provider that's dry, memory may be unchanged → skip.

☐ pass

T-BE-38 — Owner memory_consolidate returns before/after summary

- Command/where:

```
curl -s -X POST
"$BASE/api/memory_consolidate" -H
'Content-Type: application/json' \
-d '{"user":"default"}' | python3 -m
json.tool
```

- Expected: 200, `{ok, uid, before, after, removed, ...}` (server.py:6761/6780). Idempotent: a second run on already-compacted memory yields `removed:0`. No network/model call, no charge. `{"all":true}` consolidates all users (idle gate removed, server.py:8688).

☐ pass (owner-only)

T-BE-39 — Sleep-time memory job is OFF by default (document; do NOT enable)

- Command/where: read-only — confirm via code, not by setting the env var.
- Expected: the internal sleep-time consolidation job ticks **only** when `MOSTIK_SLEEP_TIME` ∈ {1,true,yes,on} (scheduler.py:50-52, `_tick_internal` no-op guard scheduler.py:68-71). Default cadence when enabled is daily 04:00 (scheduler.py:59). Leave `MOSTIK_SLEEP_TIME` UNSET. The user-facing `/api/memory Consolidate` (T-BE-38) is the manual equivalent.

☐ pass (observe-only — do not set `MOSTIK_SLEEP_TIME`)

RAG (grounding + scoping)

T-BE-40 — Doc-grounded answer cites the ingested doc

- Command/where:

```
# ingest a fact-bearing doc
curl -s -X POST "$BASE/api/rag_ingest" -H
  'Content-Type: application/json' \
  -d '{"user": "default", "name": "facts.txt", "text": "Внутренний
код проекта Тайга: ТГ-7788. Это секрет."}' >/dev/null
# ask a question that needs it (RAG auto-
injected via chat)
curl -sN -X POST "$BASE/api/chat" -H
  'Content-Type: application/json' \
  -d '{"user": "default", "model": "__auto__", "messages":
[{"role": "user", "content": "Какой внутренний код
проекта Тайга?"}]}'
# cleanup
curl -s -X POST "$BASE/api/rag_delete" -H
  'Content-Type: application/json' -d
  '{"user": "default", "name": "facts.txt"}'
>/dev/null
```

- Expected: the streamed answer contains "ТГ-7788"
— chat injects relevant doc chunks into the system prompt via `rag_context(...)` (server.py:10184). Embedding-provider dry → answer may not cite it → skip. (`/api/rag_query` from T-BE-12 is the lower-level check.)

☐ pass

T-BE-41 — Per-chat / workspace scoping

- Command/where:

```
# ingest into workspace A only
curl -s -X POST "$BASE/api/rag_ingest" -H
  'Content-Type: application/json' \
  -d '{"user":"default","name":"wsA.txt","text":"Кодовое
      слово рабочего пространства A: альфа-
      кит","workspace":"chatA"}' >/dev/null

# query in workspace A → hit
curl -s -X POST "$BASE/api/rag_query" -H
  'Content-Type: application/json' \
  -d '{"user":"default","query":"кодовое
      слово альфа-
      кит","workspace":"chatA","k":4}'

# query in workspace B → should NOT return
  the A-only doc
curl -s -X POST "$BASE/api/rag_query" -H
  'Content-Type: application/json' \
  -d '{"user":"default","query":"кодовое
      слово альфа-
      кит","workspace":"chatB","k":4}'

# cleanup
curl -s -X POST "$BASE/api/rag_delete" -H 'Content-Type:
  application/json' -d
  '{"user":"default","name":"wsA.txt","workspace":"chatA"}'
  >/dev/null
```

- Expected: query in `chatA` returns the doc; query in `chatB` does not surface the A-scoped doc (workspace/chat_id filter on ingest+query+delete, server.py:8296/8354/8380; workspace aliases (chat_id)). Global (no-workspace) docs remain visible to both.

☐ pass

Scheduler / cron

T-BE-42 — Create a time-triggered job (weekday + HH:MM)

- Command/where (via `/api/jobs`, action `add`, server.py:8580-8582):

```
curl -s -X POST "$BASE/api/jobs" -H
  'Content-Type: application/json' \
  -d '{"user":"default","action":"add","task":"Сводка новостей по
      ИИ","kind":"time","at_time":"09:00","weekdays":"weekdays"}'
  | python3 -m json.tool
```

- Expected: `{"ok":true,"job":{...}}` with `kind:"time"`, `at_time:"09:00"`, `weekdays:["mon","tue","wed","thu","fri"]`, and a future `next_run` unix ts (scheduler.py:189-214; weekday normalization scheduler.py:134-155; next-run math scheduler.py:158-186). Min interval is clamped to 600s for interval-jobs (`_MIN_INTERVAL`, scheduler.py:27/194).

☐ pass

T-BE-43 — List / toggle / delete jobs

- Command/where:

```
curl -s -X POST "$BASE/api/jobs" -H
'Content-Type: application/json' -d
'{"user":"default","action":"list"}'
# toggle (use an id from list):
{"action":"toggle","id":"
<jid>","enabled":false}
# delete:
{"action":"delete","id":"<jid>"}
```

- Expected: `list` → `{"jobs": [...]}` for that uid only (scheduler.py:217-218). `toggle` flips `enabled`; `delete` removes the job and returns the remaining list (server.py:8584-8587; scheduler.py:221-235).

☐ pass

T-BE-44 — Job fires on schedule (document the mechanism — do NOT wait live)

- Command/where: read-only / optional long-wait.
- Expected: a daemon thread ticks every 60s (`scheduler.start` → `_loop` , scheduler.py:261-273); due enabled jobs run through `_RUNNER` (= `_scheduled_runner` , wired at server.py:10829) which balance-gates non-owners (skips on \$0, server.py:10707-10709) and charges \$0.05 on completion for non-owners (server.py:10714-10715). After firing, `last_run` / `last_result` populate and `next_run` advances by the schedule. Verify by creating a job and re-listing after the trigger time (or trust the unit-tested `compute_next_run`).

☐ pass (observe-only / optional long-wait)

Billing / balances

T-BE-45 — /api/init balances: 5 wallets + total

- Command/where:

```
curl -s "$BASE/api/init?user=default" |
python3 -c 'import
sys,json;b=json.load(sys.stdin)
["balances"];print(sorted(b.keys()))'
```

- Expected: keys include the 5 wallets `venice`, `nanogpt`, `chutes`, `redpill`, `aimlapi` plus `_total_usd` (`get_balances` , server.py:2666-2700). Each wallet has `{usd, ok, ...}` + legal fields + a `low` flag (`usd < 2.0`). `_total_usd` = sum of numeric usd. (`redpill/aimlapi` carry `no_api_balance:true` — balance only on their dashboards.)

☐ pass

T-BE-46 — No-balance non-owner auto-hides paid chat models

- Command/where: compare init `models` for owner vs a fresh non-owner:

```
curl -s "$BASE/api/init?user=default" | python3 -c 'import sys,json;print("owner models",len(json.load(sys.stdin)["models"]))'
curl -s "$BASE/api/init?user=stranger123" | python3 -c 'import sys,json;print("stranger models",len(json.load(sys.stdin)["models"]))'
```

- Expected: non-owner's `models` / `full` exclude FREE chat models — the owner-only freebies are stripped for non-owners (`_free_chat_ids` , `server.py:1577`; `init filter server.py:8924-8927`; `visible_catalog_for server.py:1588`). So stranger sees fewer (only paid chat + image/voice/media). This is the "free models are owner-only" rule.

☐ pass

T-BE-47 — Free chat models are owner-only (catalog confirmation)

- Command/where:

```
curl -s "$BASE/api/catalog?user=stranger123" | python3 -c 'import sys,json;m=json.load(sys.stdin)["models"];print("free chat shown to stranger:",[x["id"] for x in m if x.get("free") and x.get("kind") not in ("image","voice","media")])'
```

- Expected: empty list — no free chat models in a non-owner's `/api/catalog` (`visible_catalog_for` , `server.py:1588-1596`). Owner's catalog (`?user=default`) DOES include them.

☐ pass

T-BE-48 — Self top-up blocked without a payment processor (non-owner)

- Command/where (with `test_topup` OFF, the default):

```
curl -s -X POST "$BASE/api/topup" -H
'Content-Type: application/json' -d
'{"user":"stranger123","rub":500}'
```

- Expected: 503, `{"error":"оплата временно недоступна — подключаем платёжную систему.", "code":"unavailable"}` — a non-owner cannot mint balance for free unless `test_topup` is on (server.py:9347-9349). Owner can always top up self for tests.

☐ pass

OpenAI-compat (Tayga SDK surface)

T-BE-49 — Mint a Mostik API key (owner)

- Command/where:

```
curl -s -X POST "$BASE/api/apikeys" -H
'Content-Type: application/json' \
-d '{"user":"default","action":"create","name":"qa-key"}
```

- Expected: `{"key":"mostik-sk-...", "keys": [...]}` — full secret returned **once** (`gen_apikey` , server.py:6366-6373; handler server.py:9297→`api_apikeys` server.py:9410). Save the `mostik-sk-...` value for the next test.

☐ pass (owner setup)

T-BE-50 — /v1/chat/completions with OpenAI-shaped body → valid completion

- Command/where (using the key from T-BE-49):

```
KEY=mostik-sk-XXXX # paste from T-BE-49
curl -s -X POST
"$BASE/v1/chat/completions" \
-H "Authorization: Bearer $KEY" -H
'Content-Type: application/json' \
-d '{"model":"__auto__","messages":
[{"role":"user","content":"Say hi
in one word"}]}' | python3 -m
json.tool
```

- Expected: 200, an OpenAI-shaped JSON completion (`choices[0].message.content` , `usage`) proxied from the upstream provider (`openai_proxy` , server.py:9419-9508). Usage is metered to the key's owner (`_charge_api` , server.py:9509). Provider dry → 502 `{error:{message}}` → skip.

☐ pass

T-BE-51 — /v1/chat/completions without/with bad key → 401

- Command/where:

```
curl -s -o /dev/null -w "%{http_code}\n" -X POST "$BASE/v1/chat/completions" \
-H 'Authorization: Bearer not-a-real-key' -H 'Content-Type: application/json' \
-d '{"model": "__auto__", "messages": [{"role": "user", "content": "hi"}]}'
```

- Expected: 401, body {"error": {"message": "Invalid Mostik API key", "type": "invalid_request_error"}} (server.py:9422-9425). Confirms the SDK surface is key-gated (separate from the uid-based internal API).

☐ pass

T-BE-52 — /v1/chat/completions enforces message-size validation → 400

- Command/where: with a valid key, send 401 messages:

```
KEY=mostik-sk-XXXX
python3 - <<PY
import json,urllib.request,urllib.error
msgs=[{"role":"user","content":"x"} for _
      in range(401)]
d=json.dumps({"model": "__auto__", "messages": msgs}).encode()
r=urllib.request.Request("http://127.0.0.1:8777/v1/chat/completions",data=d,
    headers={"Content-Type": "application/json", "Authorization": "Bearer $KEY"})
try: urllib.request.urlopen(r)
except urllib.error.HTTPError as e:
    print(e.code, e.read().decode()[:120])
PY
```

- Expected: 400, {"error": {"message": "слишком много сообщений (>400)"}} — the SDK path runs the same _sec_messages_ok guard (server.py:9433-9435).

☐ pass

T-BE-53 — Revoke the QA key (cleanup)

- Command/where: `curl -s -X POST "$BASE/api/apikeys" -H 'Content-Type: application/json' -d '{"user": "default", "action": "revoke", "id": "<id-from-keys>"}'`
- Expected: {"keys": [...]} without the revoked entry (server.py:9414-9416). Re-running T-BE-50 with the revoked key then yields 401.

☐ pass (owner cleanup)

Quick reference — verified owner-gated endpoints (file:line)

Endpoint / action	Gate site (server.py)	Non-owner result
POST /api/selftest	8593-8594	403 forbidden
POST /api/memory Consolidate	8682-8683	403 forbidden
POST /api/run (code interpreter)	9198-9199	403 forbidden
POST /api/billing (any write)	9362-9363	403 forbidden
POST /api/identity (set persona)	9204-9205	403 forbidden
POST /api/catalog_refresh	8546-8547	403 forbidden
POST /api/mcp (manage MCP)	9215-9216	403 forbidden
POST /api/browser open/act	8880-8883	429/blocked
POST /api/users rename/delete OTHER	9528-9529	403 forbidden
POST /api/topup (no processor)	9347-9349	503 unavailable
Free chat models hidden	1577/1588/8924	filtered out
Per-IP rate limit exemption	2073/7738/7756	owner exempt
Chat dev tools (shell/run_code)	10122 (and is_owner)	disabled

Owner check itself: is_owner(uid) — server.py:5213 (true iff user record has owner:true OR uid == "default").